Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

★★★★★

Université de Gabès

★★★★★

Institut Supérieur d'Informatique et de Multimédia de Gabès

Compte Rendu de Travaux Pratiques

MODULE : PROGRAMMATION EMBARQUÉES

Thème du TP : **Clignotement d'une LED sur la carte STM32F407 et Pare Feux**

Outils: Keil µVision 5 · STM32CubeMX · ST-LINK

Réalisé par :

Ibrahim Sbouai

Houssem Sadok

Mohamed Aziz Romdhan

Mohamed Yassine Touiti

Groupe : **LISI3D**

Filière : Ingénierie Systèmes Informatiques

Parcours : Systèmes Embarqués et IoT

Enseignant : **Atef Ftirich**

Date de la séance : **19/01/2026**

Année universitaire 2026 – 2027

Remerciements

Nous tenons à exprimer nos sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, au bon déroulement de cette séance de travaux pratiques et à la réalisation de ce compte rendu. Nous remercions tout particulièrement l'**Institut Supérieur d'Informatique et de Multimédia de Gabès** (**ISIMG**) pour la qualité de la formation et pour les moyens mis à notre disposition.

Nos remerciements vont à Monsieur **Atef Ftirich**, enseignant du module Programmation Embarquées, pour la clarté de ses explications, sa disponibilité et ses conseils tout au long de la séance, qui nous ont permis de mener le TP à son terme.

Nous remercions également les membres du groupe **LISI3D** pour l'esprit d'équipe et l'entraide pendant les manipulations, ainsi que l'ensemble des enseignants de l'ISIMG pour leur accompagnement.

Table des matières

Introduction générale **6**

Chapitre 1 : Généralités et plateforme STM32 ..... **7**

1. Introduction 7
2. Systèmes embarqués et microcontrôleurs 7
3. Du monde Arduino à l'écosystème STM32 7
4. Caractéristiques De La Carte STM32F4DISCOVERY 9
5. Conclusion 9

Chapitre 2 : La carte STM32F407 ..... **10**

1. Introduction 10
2. Le microcontrôleur STM32F407VGT6 10
3. Architecture interne 10
4. La carte STM32F4DISCOVERY 11
5. Les broches (GPIO) de la carte 13
6. Conclusion 16

Chapitre 3 : Outils logiciels et installation **17**

1. Introduction 17
2. STM32CubeMX 17
3. STM32CubeIDE 18
4. Keil µVision 5 19
5. ST-LINK 20
6. Tera Term 20
7. Conclusion 21

Chapitre 4 : Réalisation du TP **22**

1. Objectif et matériel 22
2. Démarche générale 22
3. Création du projet sous Keil µVision 5 22
4. Configuration avec STM32CubeMX 27
5. Ajout du fichier source et programmation 30
6. Téléchargement du programme dans la carte 33
7. Résultats obtenus 34
8. Exercice 2 : clignotement d'une LED externe (PA5) 35
9. Exercice 3 : feu tricolore (pare-feux) 37
10. Configuration du ST-Link Debugger dans Keil µVision 5 42
11. Vidéos de démonstration 43
12. Difficultés rencontrées et solutions 44
13. Conclusion 44

Conclusion générale **45**

Bibliographie **46**

1. Documents et sites de référence 46
2. Bibliothèques et logiciels : où les télécharger et comment les installer 46

Liste des figures

Figure 1 : ST-LINK/V2 intégré avec commutateur
Figure 2 : Carte STM32F407 (STM32F4DISCOVERY), vue de dessus
Figure 3 : Schéma bloc simplifie du STM32F407VGT6
Figure 4 : Disposition schématique de la carte STM32F4DISCOVERY
Figure 5 : Structure simplifiée d'une broche GPIO
Figure 6 : Page de téléchargement de STM32CubeMX sur le site de ST
Figure 7 : Assistant d'installation de STM32CubeMX
Figure 8 : Gestionnaire de packages de STM32CubeMX (STM32Cube MCU Package for STM32F4)
Figure 9 : Assistant d'installation de STM32CubeIDE
Figure 10 : Installation de Keil µVision
Figure 11 : Pack Installer de Keil µVision (pack STM32F4xx_DFP)
Figure 12 : Activation de la licence MDK-Community dans Keil µVision
Figure 13 : Pilote ST-LINK reconnu dans le Gestionnaire de périphériques
Figure 14 : Fenêtre de connexion série de Tera Term
Figure 15 : Chaîne de développement du TP
Figure 16 : Menu Project > New µVision Project dans Keil µVision 5
Figure 17 : Choix du dossier et du nom du projet (TP_LED)
Figure 18 : Fenêtre Select Device for Target (aucun composant sélectionné)
Figure 19 : Sélection de la série STM32F4 Series
Figure 20 : Sélection de STM32F407VG
Figure 21 : Sélection de la référence STM32F407VGTx
Figure 22 : Fenêtre Manage Run-Time Environment (avant sélection)
Figure 23 : Sélection de CMSIS > CORE et Device > CubeMX
Figure 24 : Projet TP_LED dans Keil et lancement de STM32CubeMX
Figure 25 : Vue Pinout & Configuration du STM32F407VGTx
Figure 26 : Choix de GPIO_Output pour la broche PD12
Figure 27 : Arbre d'horloges dans l'onglet Clock Configuration
Figure 28 : Onglet Project Manager (MDK-ARM)
Figure 29 : Copie des fichiers de bibliothèques par STM32CubeMX
Figure 30 : Confirmation de la génération du code
Figure 31 : Menu Add New Item to Group 'Source Group 1'
Figure 32 : Création du fichier main.c dans Source Group 1
Figure 33 : Code du TP dans main.c et compilation sans erreur
Figure 34 : Menu Options for File 'main.c' sur le main.c de STM32CubeMX
Figure 35 : Exclusion du main.c généré : case Include in Target Build décochée
Figure 36 : Sélection de ST-Link Debugger dans Options for Target > Debug
Figure 37 : Réglages ST-Link (Settings) et algorithme de programmation Flash
Figure 38 : Téléchargement du programme dans la Flash (Flash > Download)
Figure 39 : Résultat : LED de la carte STM32F407 qui clignote
Figure 40 : LED externe reliée à PA5 : (a) allumée, (b) éteinte
Figure 41 : Broches PA4, PA5 et PA7 configurées en GPIO_Output
Figure 42 : Arbre d'horloges (Clock Configuration)

Figure 43 : Project Manager et génération du code
Figure 44 : Code du feu tricolore dans Keil et compilation sans erreur
Figure 45 : Feu tricolore : (a) feu vert, (b) feu jaune, (c) feu rouge
Figure 46 : ST-Link Debug reconnu dans le Gestionnaire de périphériques
Figure 47 : Icônes Build et Download de Keil µVision 5

Introduction générale

Les systèmes embarqués occupent aujourd'hui une place essentielle dans notre quotidien : électroménager, automobile, objets connectés, équipements médicaux, automatisation industrielle. Au cœur de chacun de ces systèmes se trouve un microcontrôleur, un circuit intégré qui regroupe sur une seule puce un processeur, de la mémoire et des périphériques d'entrées-sorties. Savoir le configurer et le programmer est donc une compétence de base pour un futur ingénieur en systèmes embarqués et IoT.

Dans le cadre du module **Programmation Embarquées**, nous avons réalisé une séance de travaux pratiques consacrée à la carte **STM32F407**, construite autour d'un microcontrôleur STMicroelectronics à cœur ARM Cortex-M4. L'objectif de ce TP est de maîtriser la chaîne de développement complète : création d'un projet sous **Keil µVision 5**, configuration graphique des broches et de l'horloge avec **STM32CubeMX**, écriture du programme en langage C par accès direct aux registres, puis téléchargement dans la carte grâce au débogueur **ST-LINK**. L'exemple choisi est le plus simple et le plus classique en embarqué : faire clignoter une LED, puis de réaliser deux montages : une LED externe et un feu tricolore.

Ce compte rendu est organisé en quatre chapitres. Le premier présente les notions générales sur les systèmes embarqués ainsi que la plateforme Arduino (application et cartes), afin de situer le passage vers l'écosystème STM32. Le deuxième décrit en détail la carte STM32F407 : architecture interne, composants de la carte et rôle des broches. Le troisième présente les outils logiciels utilisés (STM32CubeMX, STM32CubeIDE, Keil µVision 5, Tera Term et ST-LINK) avec leur procédure d'installation pas à pas. Enfin, le quatrième chapitre détaille la réalisation du TP, illustrée par les captures d'écran des différentes étapes, avant de conclure.

Chapitre 1 : Généralités et plateforme STM32

1. Introduction

Avant d'aborder la carte STM32F407, ce chapitre rappelle ce qu'est un système embarqué, qui sert souvent de première approche de la programmation des microcontrôleurs. Cette présentation permet ensuite de comprendre pourquoi, pour des applications plus exigeantes, on se tourne vers des familles de microcontrôleurs plus puissantes comme les STM32.

2. Systèmes embarqués et microcontrôleurs

Un **système embarqué** est un système électronique et informatique conçu pour accomplir une ou plusieurs tâches précises, souvent en temps réel, et intégré dans un équipement plus large (machine à laver, voiture, capteur connecté, etc.). Contrairement à un ordinateur généraliste, il est limité en mémoire, en énergie et en coût, et son logiciel est écrit spécifiquement pour le matériel visé.

Un **microcontrôleur** (MCU) est le composant central de la majorité des systèmes embarqués. Il intègre dans un seul boîtier :

- **un processeur** (le cœur, par exemple ARM Cortex-M4) qui exécute les instructions ;
- **de la mémoire** : mémoire Flash pour stocker le programme, et mémoire SRAM pour les données ;
- **des périphériques** : ports d'entrées-sorties (GPIO), temporisateurs (timers), convertisseurs analogique-numérique (ADC), interfaces de communication (UART, SPI, I2C, USB, CAN…) ;
- **un système d'horloge et de gestion de l'alimentation.**

3. Du monde Arduino à l'écosystème STM32

Arduino est idéal pour débuter, mais ses cartes d'entrée de gamme sont limitées en puissance et en périphériques. Les microcontrôleurs STM32 de STMicroelectronics, à cœur ARM Cortex-M 32 bits, offrent des performances bien supérieures et une grande richesse de périphériques ; ils sont très utilisés dans l'industrie. Le tableau suivant compare les deux approches.

Tableau 2 : Comparaison entre une carte Arduino Uno et une carte STM32F407

Critère	STM32F407 (Discovery)
Cœur du microcontrôleur	ARM Cortex-M4F 32 bits (FPU, DSP)
Fréquence maximale	168 MHz
Mémoire Flash / SRAM	1 Mo / 192 Ko
Niveau de programmation	Accès aux registres ou bibliothèque HAL
Outils	Keil µVision, STM32CubeIDE, STM32CubeMX
Débogage	Complet via ST-LINK (points d'arrêt, pas à pas)
Niveau logique	3,3 V (certaines broches tolèrent 5 V)
Courbe d'apprentissage	Plus exigeante, mais plus formatrice

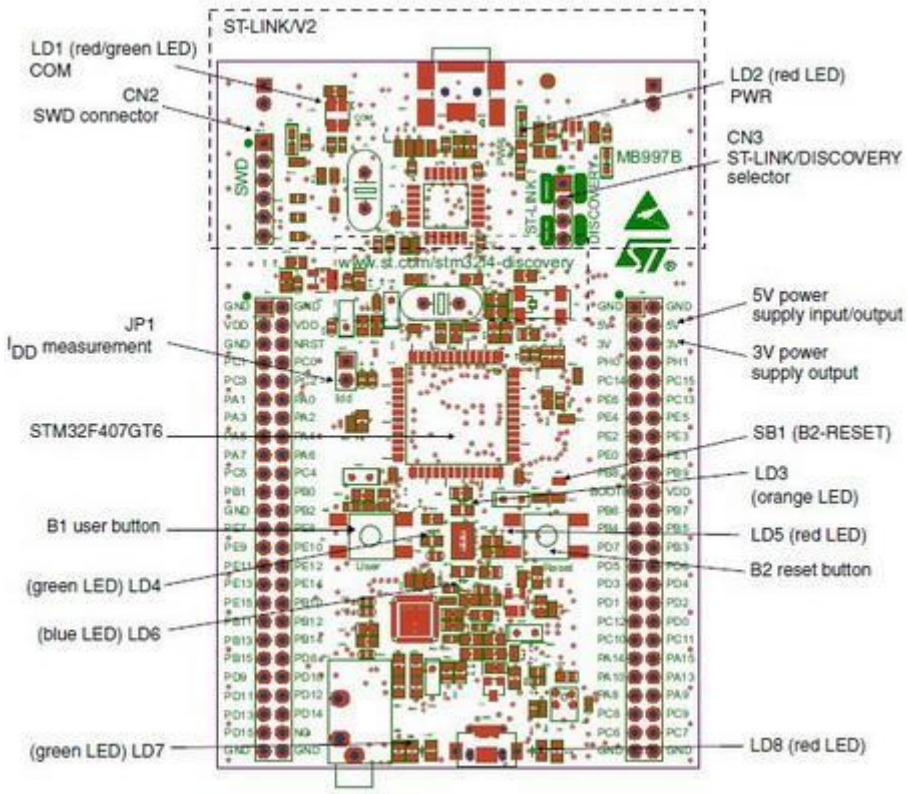


Figure 1 : ST-LINK/V2 intégré avec commutateur

4. Caractéristiques De La Carte STM32F4DISCOVERY

La carte STM32F4DISCOVERY offre les fonctionnalités suivantes :

• **Microcontrôleur STM32F407VGT6** doté d'un cœur 32 bits *Arm®(a) Cortex®-M4* avec unité de calcul en virgule flottante (**FPU**),

1 Mo de mémoire flash et 192 Ko de **RAM** dans un boîtier **LQFP100**

• **Interface USB OTG**

• Accéléromètre **MEMS** 3 axes

• **Microphone** numérique *omnidirectionnel* avec capteur audio **MEMS**

• Convertisseur numérique-analogique (**DAC**) audio avec circuit de haut-parleur intégré de classe D

• **Boutons-poussoirs** utilisateur et de réinitialisation

• Huit **LED** :

– *LD1* (**rouge**/**vert**) pour la communication USB

– *LD2* (**rouge**) pour l'alimentation 3,3 V

– Quatre LED utilisateur : *LD3* (**orange**), *LD4* (**vert**), *LD5* (**rouge**) et *LD6* (**bleu**)

– Deux LED USB OTG : *LD7* (**vert**) pour la tension du bus (**VBUS**) et *LD8* (**rouge**) pour la protection contre les surintensités

• Connecteurs :

– USB avec connecteur **Micro-AB**

• Options d'alimentation flexibles : **ST-LINK**, **USB VBUS** ou sources externes.

• Alimentation externe pour l'application : 3 V et 5 V.

• Logiciel gratuit complet incluant de nombreux exemples, intégré au package **STM32CubeF4 MCU** ou à **STSW-STM32068** pour l'utilisation des bibliothèques standard existantes.

• Débogueur/programmateur **ST-LINK/V2-A** intégré avec réénumération **USB** : stockage de masse, port **COM** virtuel et port de débogage.

• Compatibilité avec un large choix d'environnements de développement intégrés (**IDE**), notamment **IAR Embedded Workbench®**, **MDK-ARM** et **STM32CubeIDE**.

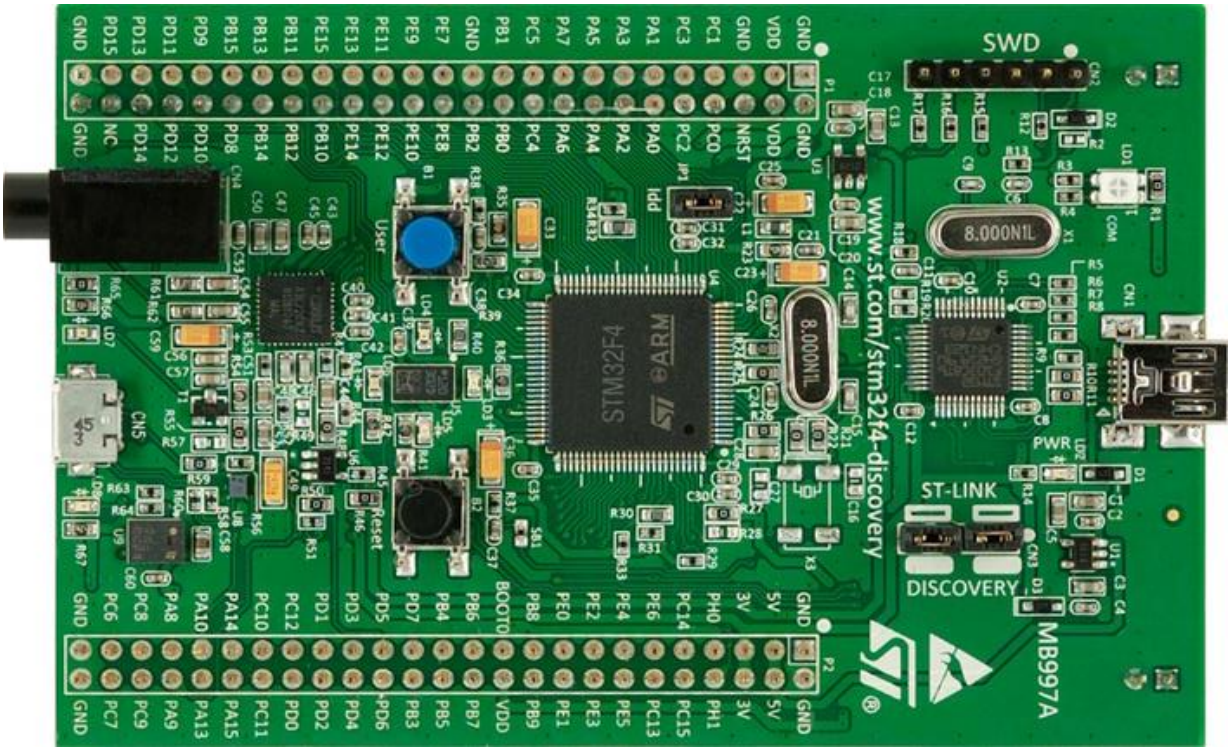


Figure 2 : STM32F407VGT6

5. Conclusion

Ce chapitre a permis de rappeler la notion de système embarqué, de définir la plateforme Arduino (application Arduino IDE et cartes) et de justifier le passage aux microcontrôleurs STM32 lorsque les besoins en performance et en périphériques augmentent. Le chapitre suivant présente en détail la carte STM32F407 utilisée dans ce TP.

Chapitre 2 : La carte STM32F407

1. Introduction

La carte utilisée en séance est construite autour du microcontrôleur **STM32F407VGT6** de STMicroelectronics. Ce chapitre décrit son architecture interne, les composants présents sur la carte (kit STM32F4DISCOVERY) et, surtout, le rôle des broches avec des exemples concrets de branchement.

2. Le microcontrôleur STM32F407VGT6

Les **STM32** forment une grande famille de microcontrôleurs 32 bits de STMicroelectronics basés sur des cœurs ARM Cortex-M. La série **STM32F4** est celle des hautes performances, avec un cœur **Cortex-M4F** (Cortex-M4 avec unité de calcul flottant FPU et instructions DSP). Le STM32F407 en est l'un des représentants les plus connus. Son nom se lit ainsi : STM32 (famille) · F (performances) · 4 (série) · 07 (ligne de produits) · V (100 broches) · G (1 Mo de Flash) · T (boîtier LQFP) · 6 (plage de température −40 à +85 °C).

Tableau 3 : Caractéristiques principales du STM32F407VGT6

Caractéristique	Valeur
Cœur	ARM Cortex-M4F 32 bits, avec FPU, instructions DSP et MPU
Fréquence maximale	168 MHz (environ 210 DMIPS)
Mémoire Flash	1 Mo (démarre à l'adresse 0x08000000)
Mémoire SRAM	192 Ko (dont 64 Ko de CCM) + 4 Ko de SRAM de sauvegarde
Boîtier	LQFP100 (100 broches), jusqu'à 82 broches d'E/S (GPIO)
Alimentation	1,8 V à 3,6 V
Timers	Jusqu'à 17 timers (16 et 32 bits), dont des timers avancés pour la PWM
Convertisseurs	3 ADC 12 bits (jusqu'à 24 canaux) et 2 DAC 12 bits
Communication	3 I2C, 4 USART + 2 UART, 3 SPI (dont 2 I2S), 2 CAN, SDIO, USB OTG FS et HS, Ethernet 10/100
Autres	DMA (16 flux), interface caméra DCMI, générateur de nombres aléatoires, contrôleur mémoire externe FSMC, RTC
Débogage	SWD et JTAG

> **Remarque :** dans Keil µVision, la description affichée lors du choix du composant (« 225 DMIPS at 180 MHz ») est celle de l'ensemble de la série STM32F4. Le STM32F407 lui-même est limité à 168 MHz.

3. Architecture interne

La figure ci-dessous donne le schéma bloc simplifié du microcontrôleur. Le cœur, la mémoire, le DMA et les périphériques communiquent par une **matrice de bus** : ils ne sont pas tous raccordés au même bus, ce qui permet des transferts simultanés et donc de bonnes performances.

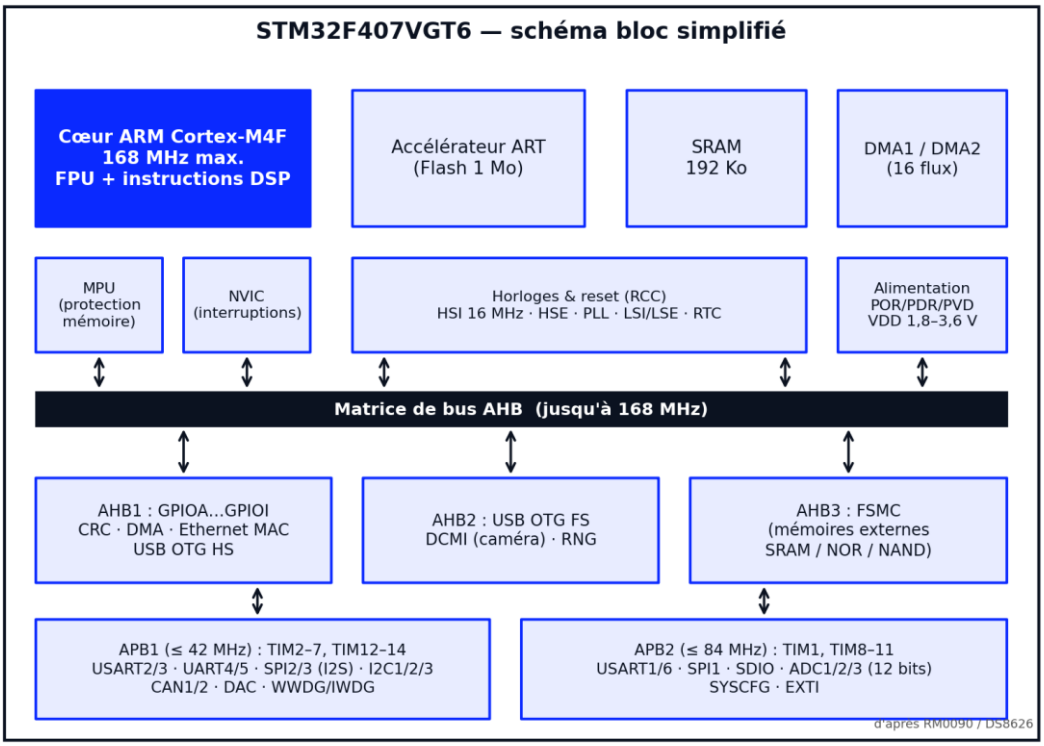


Figure 3 : Schéma bloc simplifié du STM32F407VGT6

Chaque bloc du schéma a un rôle précis :

- **Cœur Cortex-M4F** : exécute le programme. Le « F » indique la présence d'une unité de calcul flottant (FPU) et les instructions DSP accélèrent le traitement du signal.
- **MPU** (Memory Protection Unit) : protège certaines zones mémoire contre les accès non autorisés.
- **NVIC** (Nested Vectored Interrupt Controller) : gère les interruptions avec des niveaux de priorité, par exemple l'appui sur un bouton ou la fin d'une conversion ADC.
- **Accélérateur ART et mémoire Flash** : le programme est stocké en Flash ; l'accélérateur ART (préchargement et mémoire cache) permet de lire la Flash sans temps d'attente à haute fréquence.
- **SRAM** : mémoire vive qui contient les variables, la pile (stack) et le tas (heap) du programme.
- **DMA** (Direct Memory Access) : transfère des données entre périphériques et mémoire sans mobiliser le processeur.
- **RCC** (Reset and Clock Control) : génère et distribue les horloges (oscillateur interne HSI de 16 MHz, quartz externe HSE, boucle à verrouillage de phase PLL) et gère le reset. C'est lui que nous activons dans le code pour « donner l'horloge » au port GPIO.
- **Bus AHB et APB** : le bus AHB, rapide, relie les ports GPIO, le DMA, l'USB et l'Ethernet ; les bus APB1 (jusqu'à 42 MHz) et APB2 (jusqu'à 84 MHz) relient les périphériques plus lents (UART, SPI, I2C, ADC, timers).
- **Périphériques** : GPIO, timers, ADC/DAC, interfaces de communication, que le programme configure en écrivant dans leurs registres.

4. La carte STM32F4DISCOVERY

Le kit **STM32F4DISCOVERY** est une carte d'évaluation à faible coût qui permet de découvrir les fonctions du STM32F407. Elle intègre le microcontrôleur, un programmateur-débogueur ST-LINK/V2 et plusieurs composants prêts à l'emploi. La figure suivante en montre la disposition schématique.

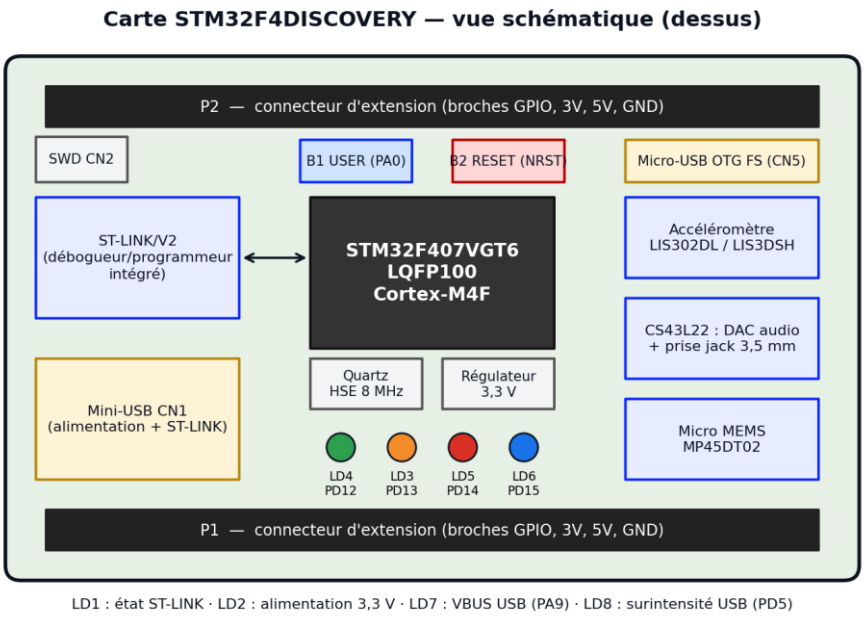


Figure 4 : Disposition schématique de la carte STM32F4DISCOVERY

Tableau 4 : Les composants de la carte et leur rôle

Composant	Rôle	Broches / liaison
Microcontrôleur STM32F407VGT6	Cerveau de la carte : exécute le programme et pilote tous les composants.	—
ST-LINK/V2 intégré	Programmateur et débogueur : relie le PC (USB) au microcontrôleur (SWD). Aucun programmateur externe n'est nécessaire.	SWDIO = PA13, SWCLK = PA14, SWO = PB3
Connecteur mini-USB (CN1)	Alimente la carte en 5 V et sert de liaison ST-LINK avec le PC.	Vers le ST-LINK
Quatre LED utilisateur	LD4 verte, LD3 orange, LD5 rouge, LD6 bleue. Une broche à l'état haut allume la LED (à travers une résistance de limitation).	PD12, PD13, PD14, PD15
Bouton utilisateur B1	Bouton poussoir lisible par le programme (niveau haut quand il est appuyé).	PA0
Bouton reset B2	Redémarre le microcontrôleur.	NRST
LED d'état (LD1, LD2)	LD1 : communication ST-LINK. LD2 : présence de l'alimentation 3,3 V.	Non programmables
USB OTG FS (micro-AB, CN5)	Permet à la carte d'être un périphérique USB (clavier, mémoire…) ou un hôte. LD7 indique VBUS et LD8 une surintensité.	PA11 (D−), PA12 (D+), PA10 (ID), PA9 (VBUS), PD5 (surintensité), PC0 (alimentation USB)
Accéléromètre 3 axes (LIS302DL ou LIS3DSH)	Capteur MEMS qui mesure l'accélération et l'inclinaison ; lu en SPI.	SPI1 : PA5 (SCK), PA6 (MISO), PA7 (MOSI), PE3 (CS) ; interruptions PE0/PE1
Microphone MEMS MP45DT02	Microphone numérique à sortie PDM.	PB10 (horloge), PC3 (données)
DAC audio CS43L22	Convertit les données audios numériques en signal analogique et amplifie vers la prise jack 3,5 mm (amplificateur classe D).	I2C1 : PB6, PB9 ; I2S3 : PC7, PC10, PC12, PA4 ; reset : PD4
Quartz 8 MHz	Horloge externe précise (HSE), multipliée par le PLL pour atteindre 168 MHz.	PH0 / PH1

Composant	Rôle	Broches / liaison
Régulateur 3,3 V	Transforme le 5 V USB en 3,3 V pour le microcontrôleur et les composants.	Broche 3V
Connecteurs P1 et P2	Rangées de broches qui exposent les GPIO, 3V, 5V et GND pour brancher des composants externes (LED, capteurs, modules).	Voir section 5

Fonctionnement du ST-LINK. Le ST-LINK est un second petit microcontrôleur placé sur la carte. Côté PC, il se présente comme un périphérique USB ; côté STM32F407, il parle le protocole **SWD** (Serial Wire Debug), qui n'utilise que deux fils (SWDIO et SWCLK). Quand Keil « télécharge » le programme, il envoie le fichier à ce ST-LINK, qui l'écrit dans la mémoire Flash du STM32F407 ; le même lien permet ensuite de déboguer (arrêter le programme, voir les variables, avancer ligne par ligne).

Fonctionnement d'une LED. Une LED laisse passer le courant dans un seul sens. Lorsque la broche de sortie est à l'état haut (3,3 V), un courant circule de la broche vers la masse à travers la résistance et la LED, qui s'allume. À l'état bas (0 V), le courant s'arrête et la LED s'éteint. Faire clignoter la LED revient donc à changer périodiquement l'état de la broche.

5. Les broches (GPIO) de la carte

5.1 Principe d'une broche GPIO

Une broche **GPIO** (General Purpose Input/Output) est une broche d'usage général que le programme peut configurer en entrée ou en sortie. Les broches sont regroupées en **ports** de 16 broches : GPIOA (PA0 à PA15), GPIOB (PB0 à PB15), et ainsi de suite jusqu'à GPIOE, plus PH0 et PH1 pour le quartz. Le nom **PD12** signifie « port D, broche 12 ». Chaque broche est contrôlée par plusieurs registres ; la figure suivante montre la structure simplifiée d'une broche.

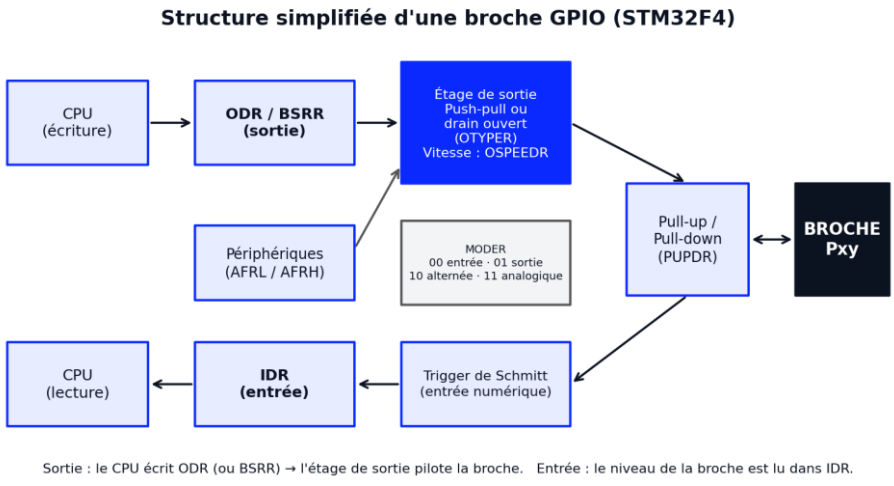


Figure 5 : Structure simplifiée d'une broche GPIO

Tableau 5 : Principaux registres d'un port GPIO

Registre	Rôle
MODER	Choix du mode de chaque broche (2 bits par broche) : 00 entrée, 01 sortie, 10 fonction alternée, 11 analogique.
OTYPER	Type de sortie : push-pull (0) ou drain ouvert (1).
OSPEEDR	Vitesse de commutation de la sortie (basse, moyenne, rapide, très rapide).

Registre	Rôle
PUPDR	Résistance interne : aucune, pull-up ou pull-down.
IDR	Lecture du niveau présent sur la broche (entrée).
ODR	Écriture du niveau de la broche (sortie).
BSRR	Mise à 1 ou à 0 d'une broche en une seule écriture, sans toucher aux autres.
AFRL / AFRH	Choix de la fonction alternée (UART, SPI, I2C, timer…) de chaque broche.

5.2 Les quatre modes d'une broche

Tableau 6 : Modes de fonctionnement d'une broche

Mode	Description	Exemple d'utilisation
Entrée (Input)	La broche lit un niveau logique (0 ou 1), avec pull-up ou pull-down optionnel.	Bouton poussoir, interrupteur, capteur à sortie logique
Sortie (Output)	La broche impose un niveau 0 ou 1, en push-pull ou drain ouvert.	LED, buzzer, relais (via transistor)
Fonction alternée	La broche est confiée à un périphérique interne (USART, SPI, I2C, timer…).	PA2/PA3 en USART2 pour un module Bluetooth ; PWM d'un servomoteur
Analogique	La broche est reliée à l'ADC (lecture d'une tension) ou au DAC (génération d'une tension).	Potentiomètre, capteur de température LM35

5.3 Les broches par port

Le tableau suivant résume les fonctions principales de chaque port. La liste complète des fonctions alternées de chaque broche se trouve dans la fiche technique (datasheet) du STM32F407, tableau « Alternate function mapping ». Le logiciel STM32CubeMX affiche aussi ces fonctions en cliquant sur une broche.

Tableau 7 : Fonctions principales des ports du STM32F407

Port	Broches	Fonctions principales (exemples)
GPIOA	PA0 à PA15	PA0 à PA7 : entrées analogiques ADC (IN0 à IN7). PA0 : bouton B1, TIM2. PA2/PA3 : USART2 (TX/RX). PA4/PA5 : sorties DAC1/DAC2. PA4 à PA7 : SPI1. PA8 : sortie d'horloge MCO1, I2C3_SCL. PA9/PA10 : USART1 (TX/RX). PA11/PA12 : USB et CAN1. PA13/PA14 : SWD (débogage). PA15 : JTDI.
GPIOB	PB0 à PB15	PB0/PB1 : ADC (IN8/IN9) et timer TIM3. PB3 : SWO. PB6/PB7 : I2C1 (SCL/SDA), USART1, TIM4. PB8/PB9 : I2C1, CAN1. PB10/PB11 : USART3, I2C2. PB12 à PB15 : SPI2.
GPIOC	PC0 à PC15	PC0 à PC5 : entrées ADC (IN10 à IN15). PC6/PC7 : USART6, TIM3/TIM8. PC8 à PC12 et PD2 : interface carte SD (SDIO). PC10 à PC12 : SPI3, UART4/5. PC13 : RTC. PC14/PC15 : quartz 32 kHz.
GPIOD	PD0 à PD15	PD0/PD1 : CAN2. PD2 : SDIO, UART5. PD5/PD6 : USART2. PD8/PD9 : USART3. PD12 à PD15 : timer TIM4 (voies 1 à 4) et LED de la carte. Nombreuses broches du bus FSMC.
GPIOE	PE0 à PE15	PE0/PE1 : TIM4, interruptions de l'accéléromètre. PE3 : sélection (CS) de l'accéléromètre. PE9, PE11, PE13, PE14 : voies du timer TIM1 (PWM). PE7 à PE15 : bus FSMC.
PH0 / PH1	2 broches	Quartz externe HSE (OSC_IN / OSC_OUT).

5.4 Les broches spéciales

Tableau 8 : Broches spéciales du boîtier LQFP100

Broche	Rôle
VDD / VSS	Alimentation 3,3 V du microcontrôleur et masse (plusieurs paires sur le boîtier).
VDDA / VSSA / VREF+	Alimentation et référence de tension des convertisseurs analogiques (ADC/DAC).
VBAT	Alimentation de secours (pile) pour la RTC et la SRAM de sauvegarde.
VCAP_1 / VCAP_2	Condensateurs du régulateur de tension interne.
NRST	Reset (actif à l'état bas) : relié au bouton B2.
BOOT0 / PB2 (BOOT1)	Choix du mode de démarrage : Flash, mémoire système (bootloader) ou SRAM.
PH0 / PH1	Entrée et sortie du quartz 8 MHz.
PA13 / PA14 / PB3	Liaison de débogage SWD (SWDIO, SWCLK) et trace SWO : à ne pas réaffecter sous peine de perdre la liaison avec le ST-LINK.

5.5 Broches déjà utilisées par la carte

Sur la carte Discovery, plusieurs broches sont déjà reliées à des composants. On peut les utiliser, mais il faut savoir que le composant correspondant est lui aussi branché dessus.

Tableau 9 : Broches déjà occupées sur la carte

Broches	Utilisation sur la carte
PD12, PD13, PD14, PD15	LED LD4 (verte), LD3 (orange), LD5 (rouge), LD6 (bleue)
PA0	Bouton utilisateur B1
PA5, PA6, PA7, PE3	Accéléromètre (SPI1) : horloge, données et sélection
PB6, PB9, PC7, PC10, PC12, PA4, PD4	Codec audio CS43L22 (I2C1, I2S3, reset)
PB10, PC3	Microphone MEMS
PA9, PA10, PA11, PA12, PD5, PC0	USB OTG FS (VBUS, ID, D−, D+, surintensité, alimentation)
PA13, PA14, PB3	Débogage SWD / SWO
PH0, PH1	Quartz 8 MHz

> **Attention :** la broche **PA5**, utilisée dans l'exemple de code du TP, fait partie de la liaison SPI de l'accéléromètre sur la carte Discovery. Sur cette carte, la LED utilisateur verte LD4 est sur **PD12**. Voir la remarque à la section 5 du chapitre 4.

5.6 Exemples de branchement

Le tableau suivant propose des exemples de composants que l'on peut brancher sur des broches du connecteur P1/P2, avec le mode à choisir dans STM32CubeMX. Avant de câbler, il faut vérifier dans le manuel de la carte (UM1472) que la broche choisie est bien libre.

Tableau 10 : Exemples de composants à brancher sur les broches

Composant	Broche (exemple)	Mode à configurer	Remarque
LED externe	PE7	GPIO_Output	Résistance de 220 Ω en série ; anode côté broche, cathode vers GND.
Bouton poussoir	PB12	GPIO_Input + pull-up	Un côté du bouton à la broche, l'autre à GND ; lecture à 0 quand il est appuyé.

Composant	Broche (exemple)	Mode à configurer	Remarque
Potentiomètre	PA1	Analog (ADC1_IN1)	Extrémités sur 3V et GND, curseur sur la broche (ne pas dépasser 3,3 V).
Servomoteur	PB0	TIM3_CH3 (PWM)	Signal PWM de 50 Hz ; alimentation du servo en 5 V sur la broche 5V.
Module Bluetooth HC-05	PA2 (TX) / PA3 (RX)	USART2	TX du module vers RX du STM32 et inversement (croisé).
Écran OLED ou capteur I2C	PA8 (SCL) / PC9 (SDA)	I2C3	Résistances de pull-up de 4,7 kΩ si elles ne sont pas sur le module.
Buzzer actif	PE9	GPIO_Output ou TIM1_CH1	Une sortie logique suffit pour un buzzer actif.

Règles à respecter : les broches fonctionnent en **3,3 V** (ne jamais leur appliquer plus de 3,3 V sauf broches tolérantes 5 V indiquées dans la datasheet) ; le courant maximal par broche est d'environ 25 mA, d'où l'usage obligatoire d'une résistance avec une LED ; pour une charge plus puissante (moteur, relais), on passe par un transistor.

6. Conclusion

Ce chapitre a présenté le microcontrôleur STM32F407VGT6, son architecture interne, les composants de la carte STM32F4DISCOVERY et le fonctionnement des broches GPIO, avec des exemples de branchement. Ces connaissances sont nécessaires pour comprendre la configuration réalisée dans le TP. Le chapitre suivant présente les logiciels utilisés et leur installation.

Chapitre 3 : Outils logiciels et installation

1. Introduction

Pour programmer la carte STM32F407, plusieurs outils complémentaires sont nécessaires. Ce chapitre définit chacun d'eux, explique son rôle et donne la procédure d'installation étape par étape. Les noms exacts des boutons peuvent légèrement varier selon la version du logiciel.

Tableau 11 : Outils utilisés et rôle de chacun

Outil	Type	Rôle dans le TP
STM32CubeMX	Configurateur graphique	Configure les broches, l'horloge et génère le code d'initialisation.
STM32CubeIDE	Environnement de développement	Éditeur, compilateur et débogueur de STMicroelectronics (alternative à Keil).
Keil µVision 5	Environnement de développement	Création du projet, édition du code C, compilation et téléchargement dans la carte.
ST-LINK	Programmateur-débogueur et pilote	Liaison USB entre le PC et le microcontrôleur pour télécharger et déboguer.
Tera Term	Terminal série	Affiche les données reçues par une liaison série (UART).

2. STM32CubeMX

2.1 Définition

STM32CubeMX est l'outil graphique de STMicroelectronics pour configurer un microcontrôleur STM32 sans écrire les registres à la main : on choisit le composant, on clique sur les broches pour leur donner une fonction, on règle l'arbre d'horloges, puis l'outil génère automatiquement le code C d'initialisation (et les fichiers du projet pour Keil, STM32CubeIDE ou IAR). Il fonctionne aussi comme composant intégré à Keil µVision.

2.2 Installation pas à pas

1. Ouvrir le site de STMicroelectronics (**https://www.st.com/en/development-tools/stm32cubemx.html**) et cliquer sur **Get Software**.
2. Se connecter avec un compte ST (en créer un gratuitement si nécessaire), accepter le contrat de licence, puis télécharger le fichier d'installation pour Windows.
3. Extraire l'archive téléchargée, puis lancer le programme d'installation **SetupSTM32CubeMX** en **clic droit > Exécuter en tant qu'administrateur**.
4. Dans l'assistant, cliquer sur **Next**, accepter la licence, choisir le dossier d'installation (garder le chemin par défaut, sans caractères spéciaux), puis **Next** jusqu'à **Finish**.
5. Lancer STM32CubeMX. Au premier démarrage, ouvrir **Help > Manage embedded software packages**, déplier **STM32F4** et installer la dernière version de **STM32Cube MCU Package for STM32F4** (la connexion à un compte ST est demandée).
6. Vérifier les mises à jour avec **Help > Check for updates**.

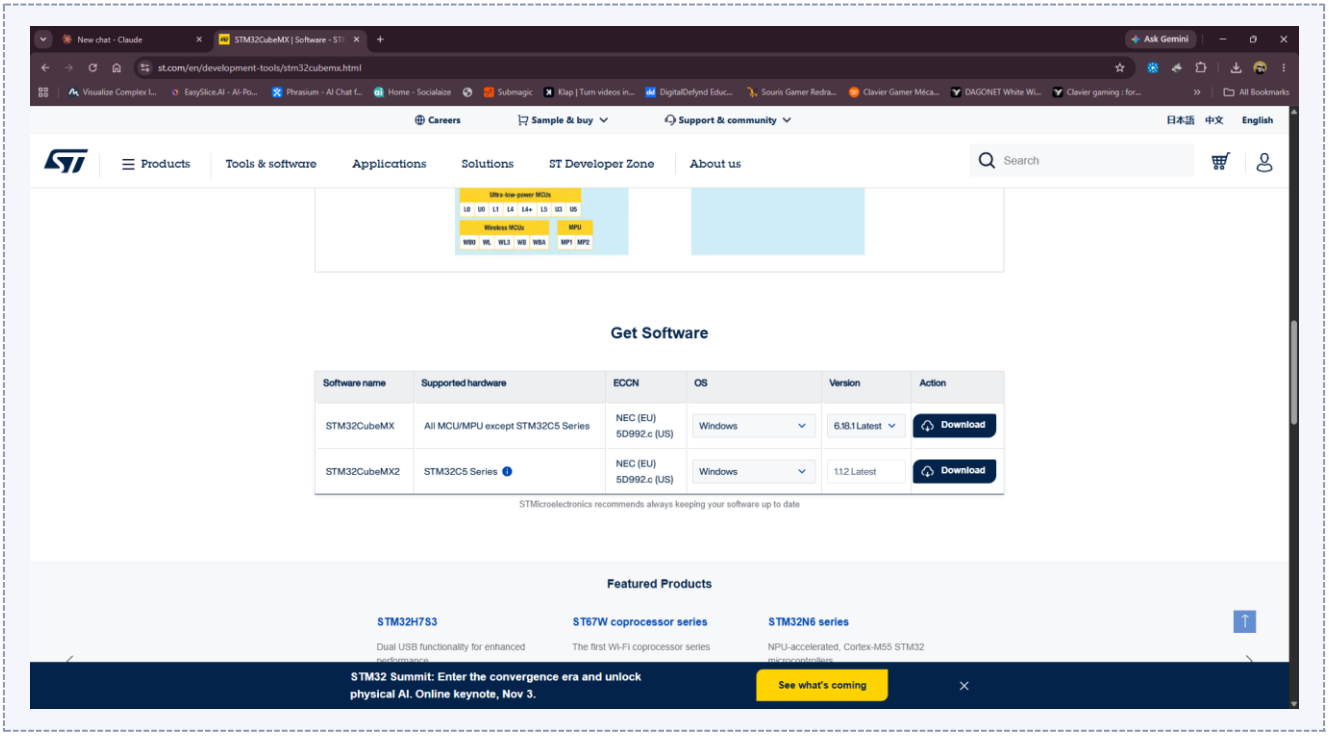


Figure 6 : Page de téléchargement de STM32CubeMX sur le site de ST

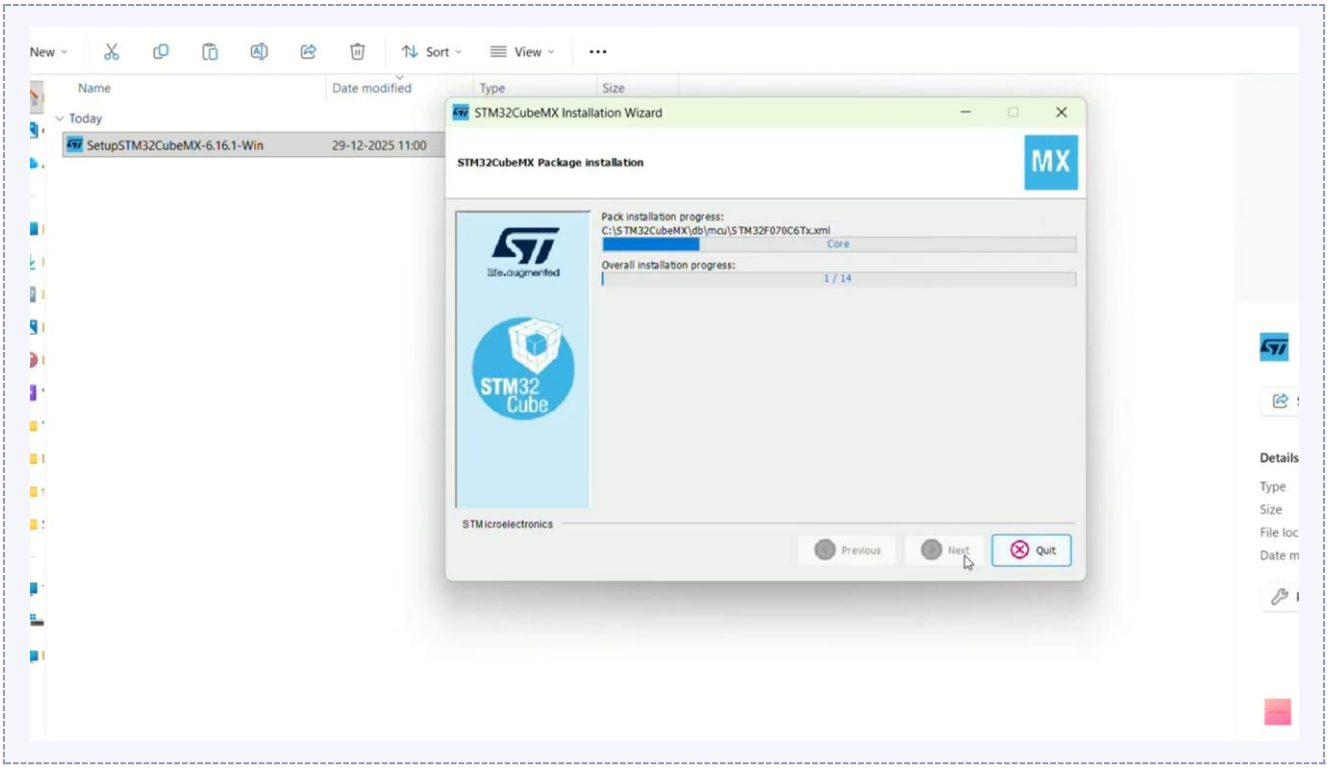


Figure 7 : Assistant d'installation de STM32CubeMX

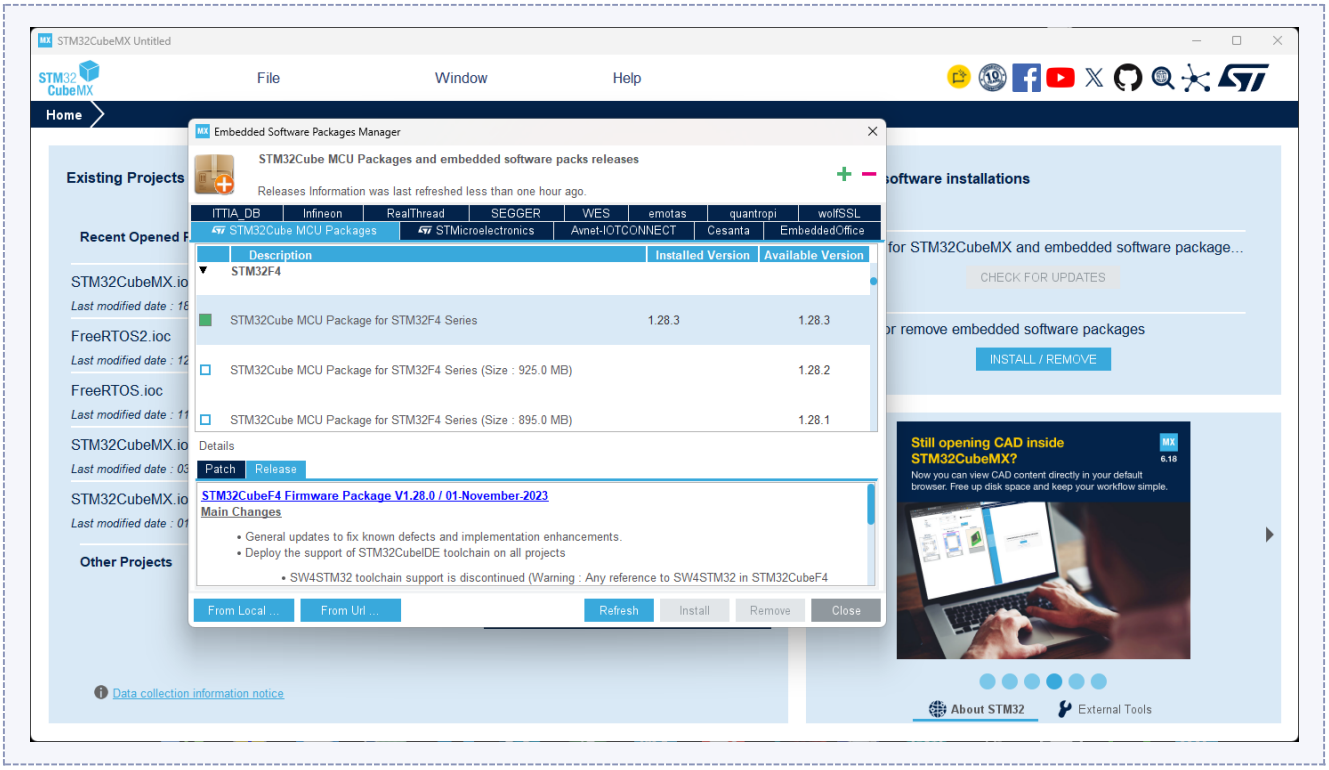


Figure 8 : Gestionnaire de packages de STM32CubeMX (STM32Cube MCU Package for STM32F4)

3. STM32CubeIDE

3.1 Définition

STM32CubeIDE est l'environnement de développement intégré gratuit de STMicroelectronics, basé sur Eclipse. Il rassemble l'éditeur de code, le compilateur GCC et le débogueur pour les STM32. Depuis la version 2.0, le module STM32CubeMX n'est plus intégré à l'intérieur de STM32CubeIDE : il faut utiliser **STM32CubeMX en version autonome** pour créer le projet et ouvrir les fichiers .ioc. Dans ce TP, nous utilisons Keil µVision 5, mais STM32CubeIDE reste une alternative : il suffit de choisir **STM32CubeIDE** dans le champ Toolchain / IDE du Project Manager de STM32CubeMX.

3.2 Installation pas à pas

1. Ouvrir la page **https://www.st.com/en/development-tools/stm32cubeide.html** et cliquer sur **Get Software** pour la version Windows ; se connecter avec son compte ST.
2. Extraire l'archive téléchargée, puis lancer l'installateur de STM32CubeIDE en tant qu'administrateur.
3. Suivre l'assistant : **Next**, acceptation de la licence, choix du dossier d'installation (chemin court, sans espace ni accent de préférence).
4. Laisser coché l'installation des pilotes **ST-LINK** (et SEGGER J-Link si proposé), puis confirmer l'installation des pilotes quand Windows le demande.
5. Lancer STM32CubeIDE, choisir un dossier de travail (**workspace**) et attendre la fin de l'initialisation.

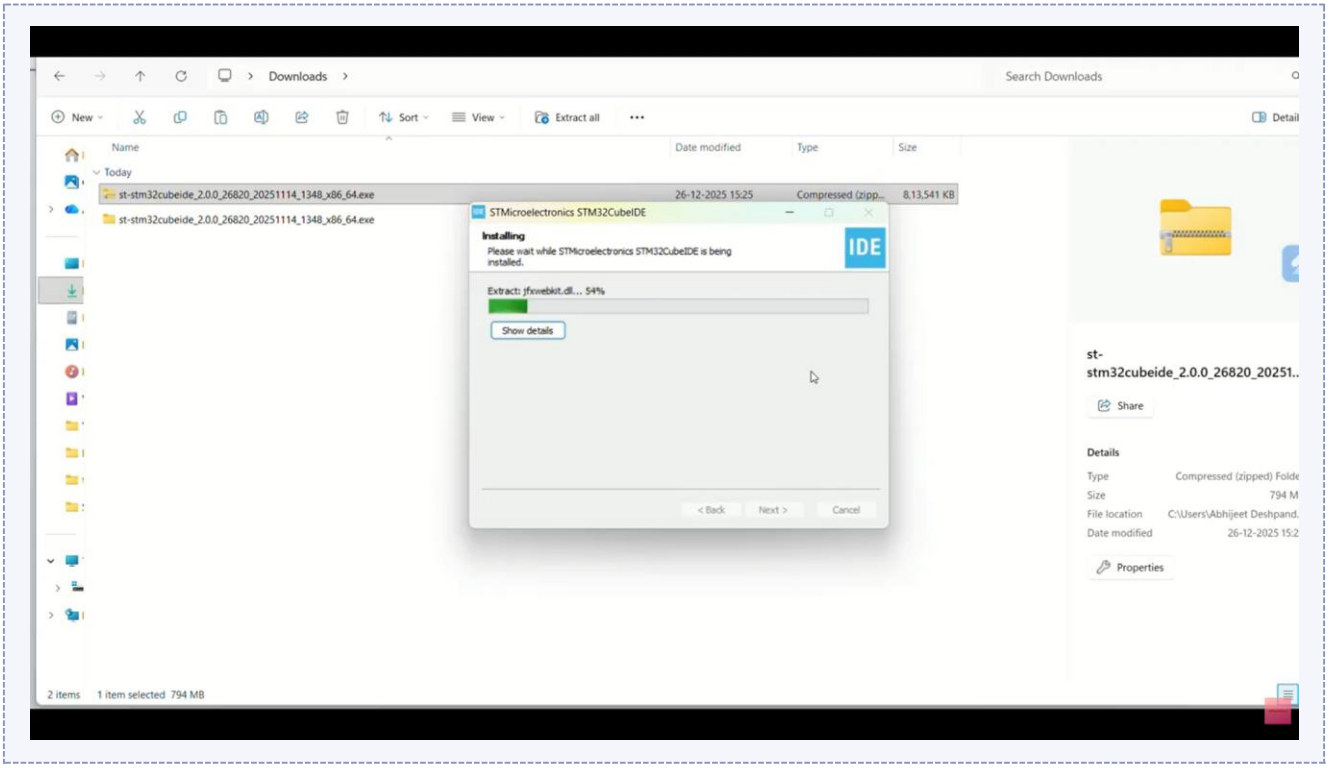


Figure 9 : Assistant d'installation de STM32CubeIDE

4. Keil µVision 5

4.1 Définition

Keil µVision est l'environnement de développement d'Arm (anciennement Keil) pour les microcontrôleurs ARM Cortex-M. Il réunit l'éditeur, le compilateur ARM, l'assembleur, l'éditeur de liens, le simulateur et le débogueur. Son gestionnaire de **packs logiciels** (Pack Installer) permet d'ajouter la prise en charge de chaque famille de microcontrôleurs. L'édition **MDK-Community** est gratuite pour les projets non commerciaux.

4.2 Installation pas à pas

1. Ouvrir **https://www.keil.arm.com/**, choisir **Keil µVision** puis **Download Keil µVision** (un formulaire d'inscription peut être demandé).
2. Lancer le programme d'installation téléchargé, cliquer sur **Next**, accepter la licence et choisir le dossier d'installation.
3. Renseigner le nom et l'adresse e-mail demandés, puis cliquer sur **Next** et **Finish**.
4. À la fin, le **Pack Installer** s'ouvre : dans **Devices > STMicroelectronics > STM32F4 Series > STM32F407**, installer le pack de la famille **STM32F4xx_DFP** ainsi que **ARM::CMSIS** (bouton **Install**). Le pack se télécharge depuis Internet.
5. Activer la licence gratuite : dans µVision, **File > License Management…**, onglet **User-Based License**, cliquer sur **License Server…**, saisir l'adresse **https://mdk-preview.keil.arm.com**, cliquer sur **Query** puis sélectionner **Keil MDK Community**.
6. Fermer la fenêtre : µVision est prêt. On peut vérifier l'édition dans **Help > About**.

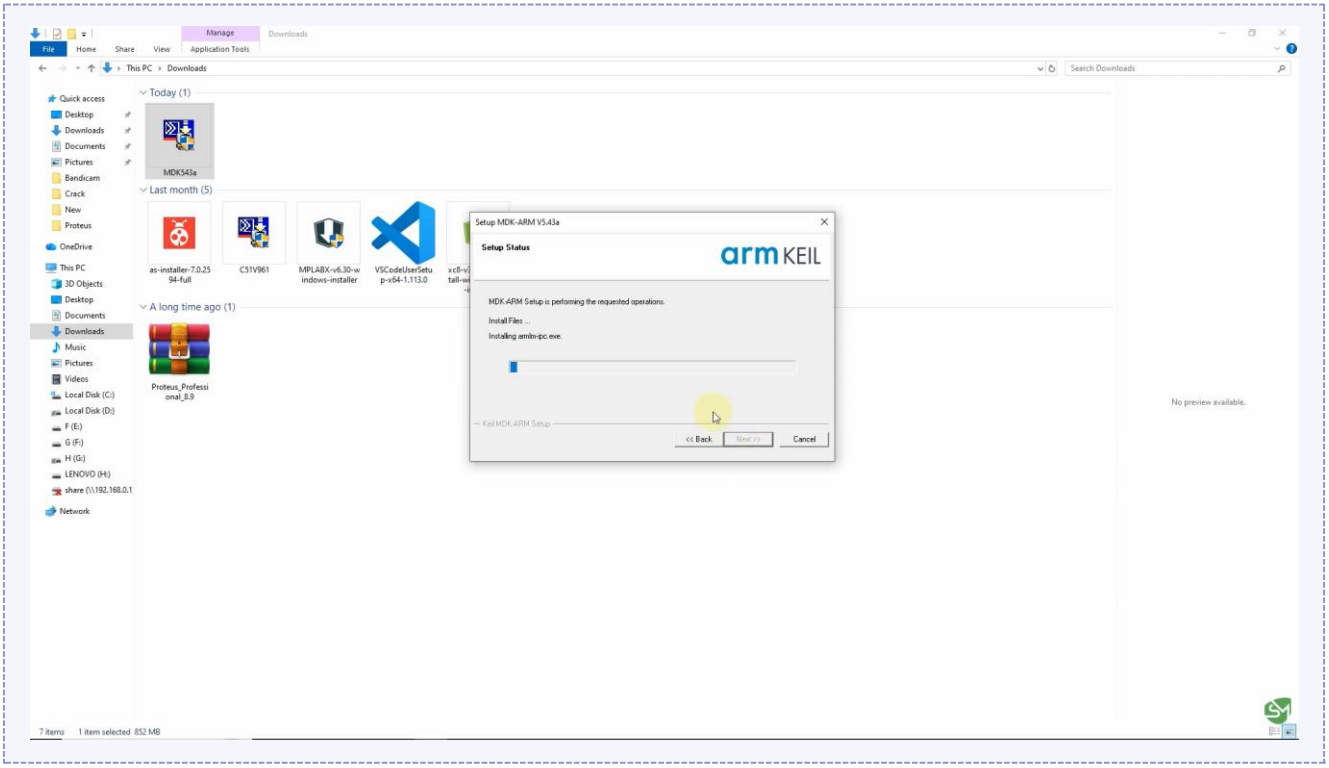


Figure 10 : Installation de Keil µVision

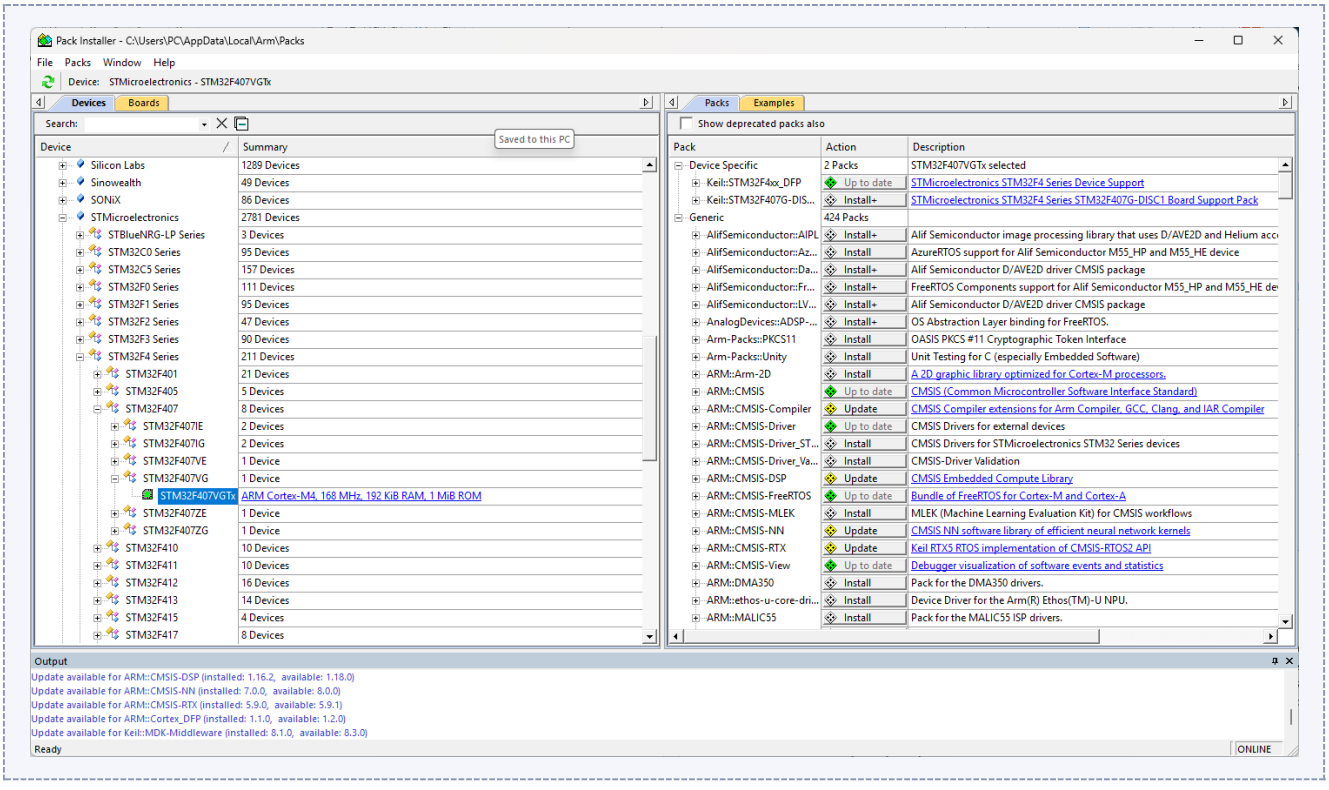


Figure 11 : Pack Installer de Keil µVision (pack STM32F4xx_DFP)

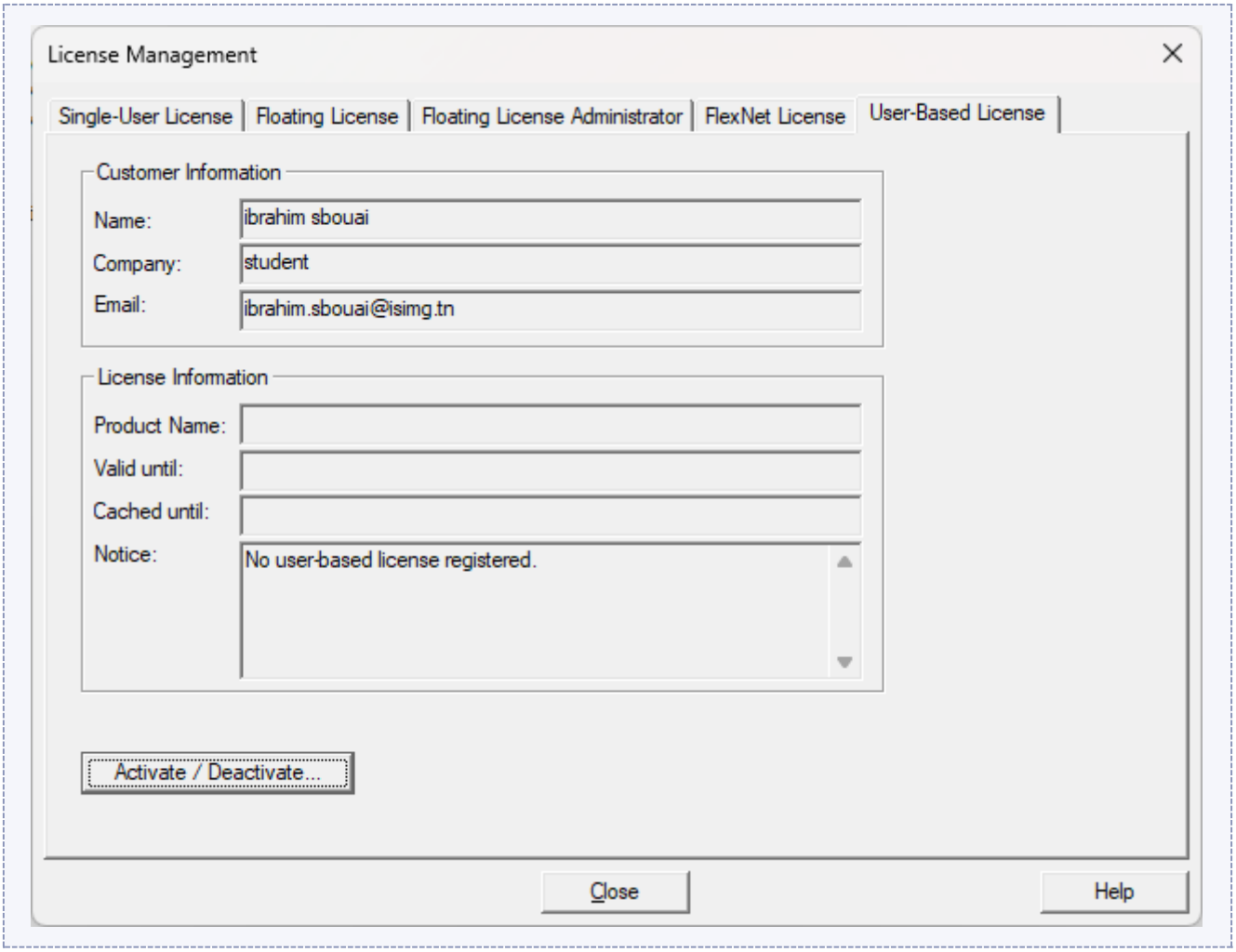


Figure 12 : Activation de la licence MDK-Community dans Keil µVision

5. ST-LINK

5.1 Définition

ST-LINK est le système de programmation et de débogage de STMicroelectronics pour les STM32. Sur la carte STM32F4DISCOVERY, un ST-LINK/V2 est déjà intégré : il suffit de relier la carte au PC avec un câble USB (type A vers mini-B) pour pouvoir télécharger le programme et déboguer. Pour que Windows reconnaisse le ST-LINK, il faut installer le pilote **STSW-LINK009** (il est aussi installé automatiquement avec STM32CubeIDE).

5.2 Installation du pilote pas à pas

1. Ouvrir **https://www.st.com/en/development-tools/stsw-link009.html** et cliquer sur **Get Software** (connexion au compte ST ; le lien de téléchargement peut être envoyé par e-mail).
2. Extraire l'archive téléchargée.
3. **Avant de brancher la carte**, lancer en administrateur le programme d'installation du pilote (le fichier **stlink_winusb_install.bat** ou l'exécutable **dpinst_amd64.exe** pour un Windows 64 bits).
4. Brancher ensuite la carte au PC avec le câble USB sur le connecteur mini-USB de la carte.
5. Ouvrir le **Gestionnaire de périphériques** de Windows et vérifier qu'un périphérique **STM32 STLink** (ou ST-Link Debug) apparaît dans la liste des périphériques USB, sans point d'exclamation jaune.
6. Sur la carte, la LED LD1 clignote en rouge puis passe au vert lorsque la communication avec Keil est établie.

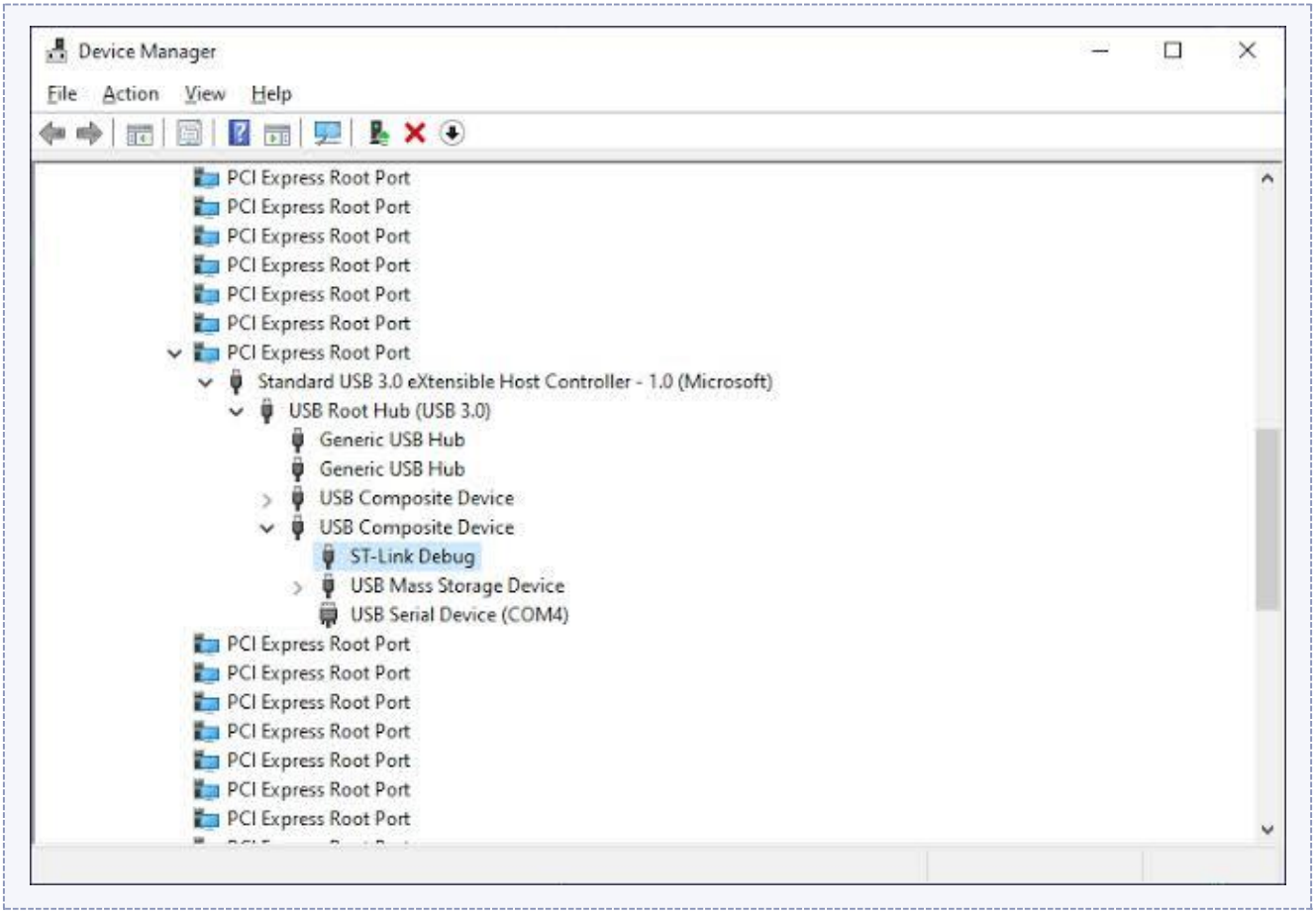


Figure 13 : Pilote ST-LINK reconnu dans le Gestionnaire de périphériques

6. Tera Term

6.1 Définition

Tera Term est un émulateur de terminal gratuit et libre pour Windows. Il permet de se connecter à un port série (COM) afin d'afficher les messages envoyés par un microcontrôleur via une liaison UART, ou de lui envoyer des commandes depuis le clavier. C'est l'équivalent du moniteur série de l'Arduino IDE.

6.2 Installation et utilisation pas à pas

1. Télécharger l'installateur depuis la page officielle du projet : **https://github.com/TeraTermProject/teraterm/releases** (fichier **teraterm-x.xx-setup.exe**).
2. Lancer l'installateur, choisir la langue, accepter la licence, garder les composants par défaut puis le dossier d'installation, et cliquer sur **Install** puis **Finish**.
3. Brancher l'adaptateur USB-série (ou le module de communication) au PC et noter son numéro de port COM dans le Gestionnaire de périphériques.
4. Lancer Tera Term : dans la fenêtre **New connection**, choisir **Serial** puis le port COM correspondant.
5. Ouvrir **Setup > Serial port…** et régler : vitesse **115200**, données **8 bits**, parité **none**, arrêt **1 bit**, contrôle de flux **none** (les mêmes valeurs que dans le programme du microcontrôleur).

> **Remarque :** le ST-LINK de la carte STM32F4DISCOVERY n'offre pas de port COM virtuel. Pour afficher des messages dans Tera Term, il faut donc relier un adaptateur USB-série aux broches d'un USART (par exemple PA9/PA10 pour USART1). Tera Term n'est pas indispensable pour le TP de la LED, mais il sera utile pour les prochains TP sur la communication série.

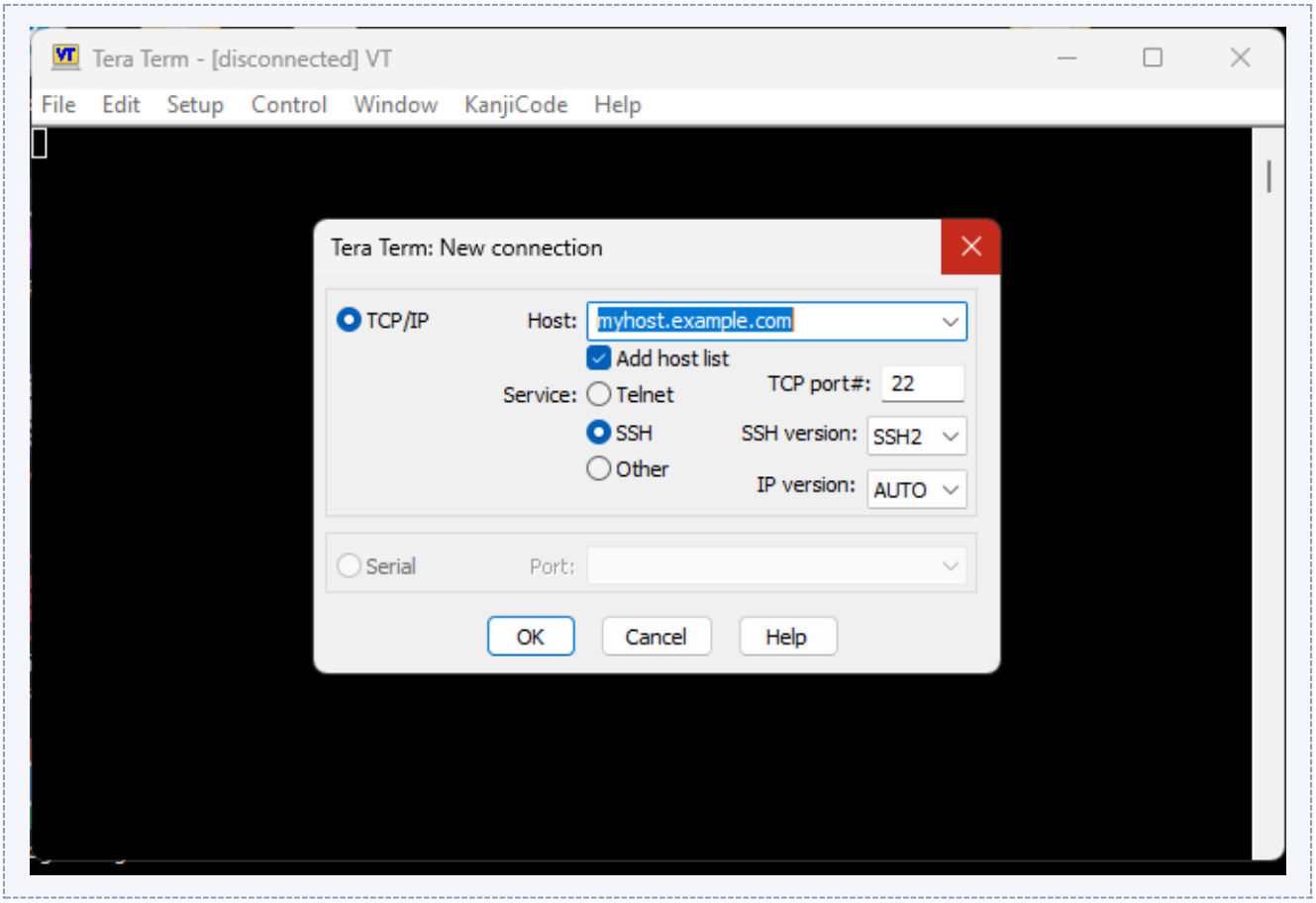


Figure 14 : Fenêtre de connexion série de Tera Term

7. Conclusion

Les cinq outils présentés se complètent : STM32CubeMX configure le microcontrôleur, Keil µVision (ou STM32CubeIDE) compile le programme, ST-LINK le transfère dans la carte et Tera Term permet d'observer la communication série. Avec ces outils installés, nous pouvons réaliser le TP.

Chapitre 4 : Réalisation du TP

1. Objectif et matériel

L'objectif du TP est de **faire clignoter une LED** de la carte STM32F407 en créant un projet sous Keil µVision 5, en configurant la broche **PD12** en sortie avec STM32CubeMX, puis en programmant l'inversion périodique de l'état de la LED en langage C. Le tableau suivant liste le matériel et les logiciels utilisés.

Tableau 12 : Matériel et logiciels du TP

Élément	Détail
Carte	STM32F407 (kit STM32F4DISCOVERY, microcontrôleur STM32F407VGTx)
Liaison PC–carte	Câble USB (type A vers mini-B) sur le connecteur du ST-LINK
Logiciels	Keil µVision 5 (MDK-Community), STM32CubeMX, pilote ST-LINK
Système	Windows 11 (64 bits)
Langage	C, accès direct aux registres (fichier d'en-tête stm32f407xx.h)

2. Démarche générale

La figure suivante résume la chaîne de développement suivie. Les étapes sont détaillées dans les sections suivantes.

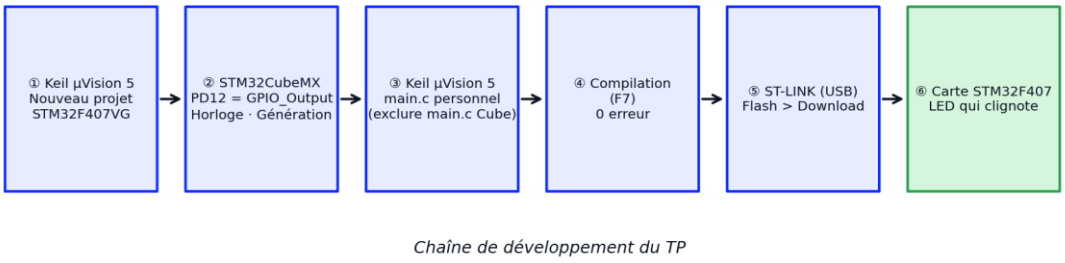


Figure 15 : Chaîne de développement du TP

3. Création du projet sous Keil µVision 5

3.1 Nouveau projet et choix du microcontrôleur

On lance Keil µVision 5, puis on ouvre le menu **Project** et on choisit **New µVision Project…**.

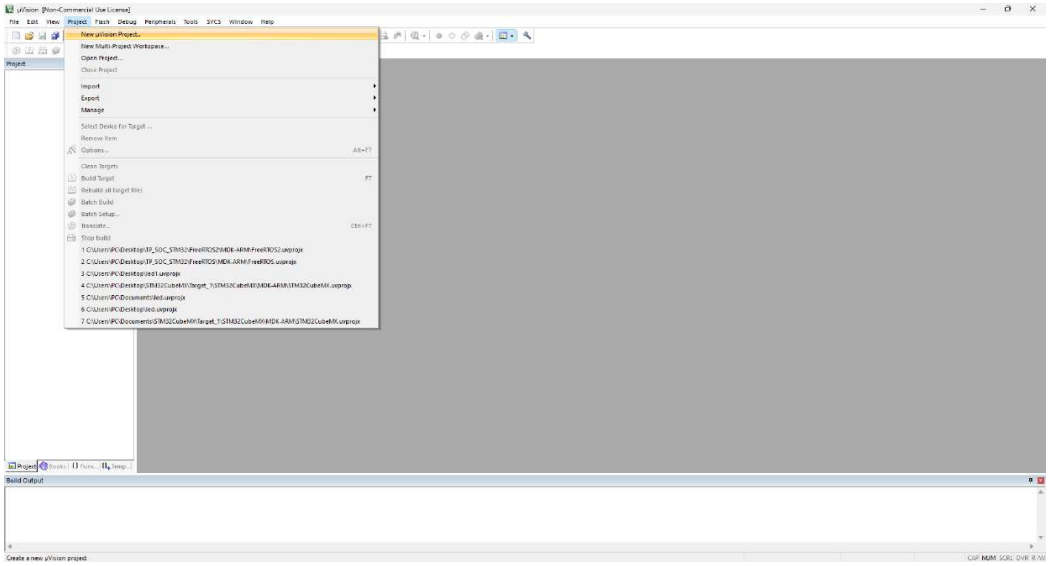


Figure 16 : Menu Project > New µVision Project dans Keil µVision 5

Une fenêtre d'enregistrement s'ouvre. On sélectionne le dossier de travail du TP (ici « STM32 TP AT » sur le Bureau), on saisit le nom du projet **TP_LED** et on clique sur **Save**. Keil crée alors le fichier de projet de type .uvprojx.

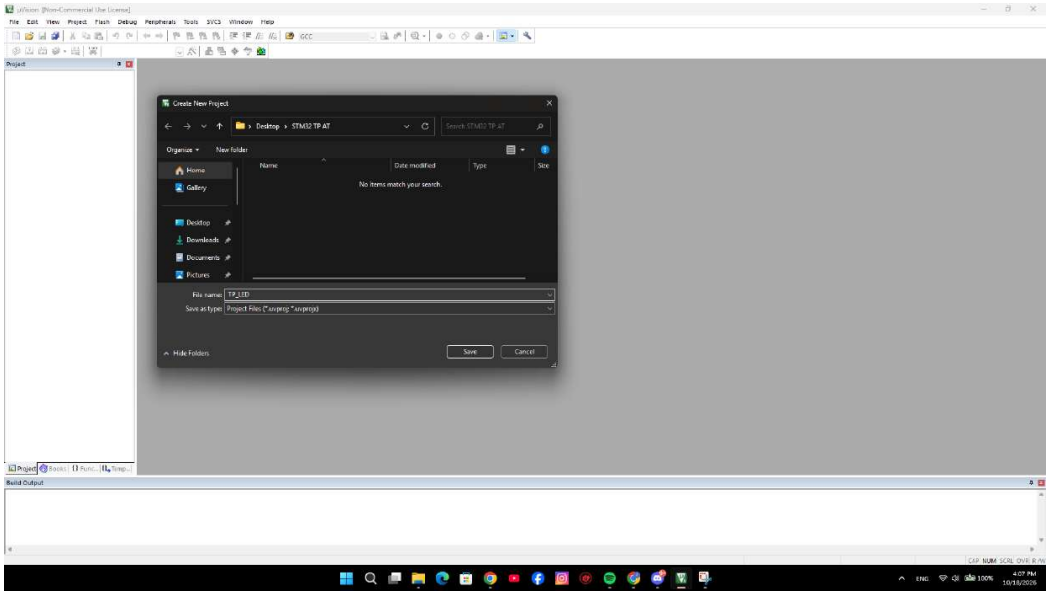


Figure 17 : Choix du dossier et du nom du projet (TP_LED)

La fenêtre **Select Device for Target** apparaît ensuite. Elle sert à indiquer à Keil le microcontrôleur de la carte, afin qu'il utilise le bon compilateur, les bons fichiers de démarrage et le bon algorithme de programmation de la Flash. Au départ, aucun composant n'est sélectionné : les champs Vendor, Device et Toolset affichent « unknown ».

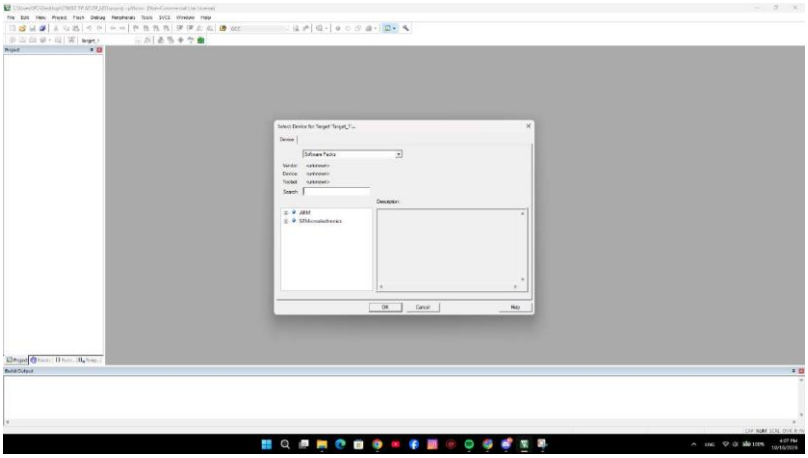


Figure 18 : Fenêtre Select Device for Target (aucun composant sélectionné)

On déplie le fabricant **STMicroelectronics**, puis la série **STM32F4 Series**, qui regroupe les microcontrôleurs STM32F401, F405, F407, F410, etc.

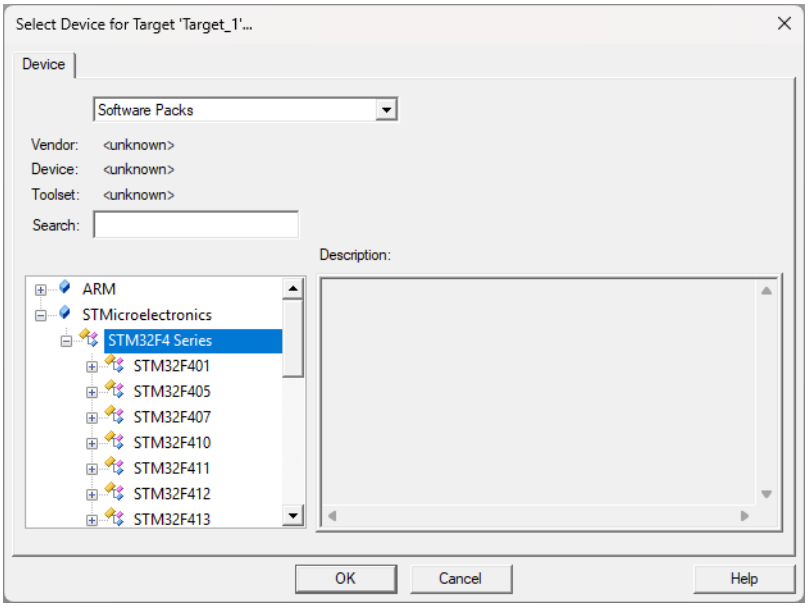


Figure 19 : Sélection de la série STM32F4 Series

On déplie **STM32F407** et on choisit **STM32F407VG** (le « V » indique le boîtier 100 broches et le « G » la Flash de 1 Mo). La description affichée est celle de la famille : cœur Cortex-M4 avec MPU, DSP et FPU.

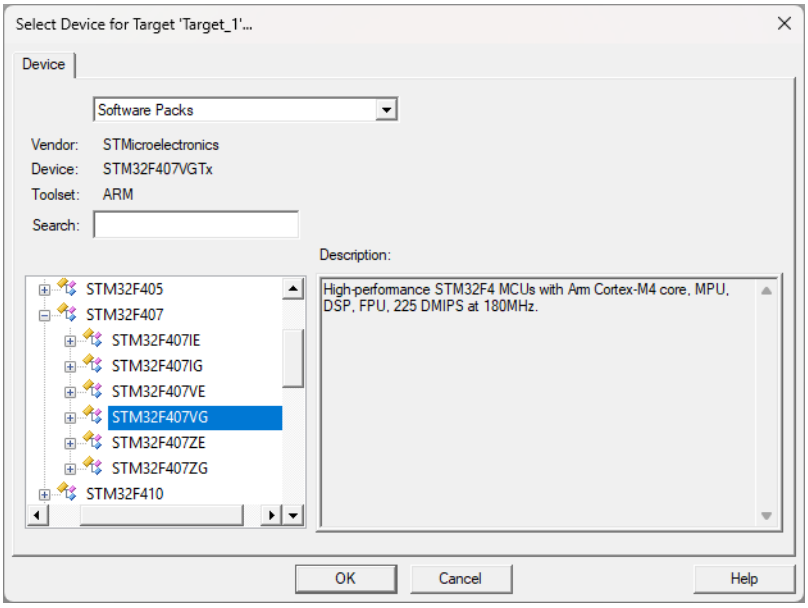


Figure 20 : Sélection de STM32F407VG

En dépliant STM32F407VG, on sélectionne la référence exacte **STM32F407VGTx**. Les champs affichent alors Vendor : STMicroelectronics, Device : STM32F407VGTx et Toolset : ARM. On valide avec **OK**.

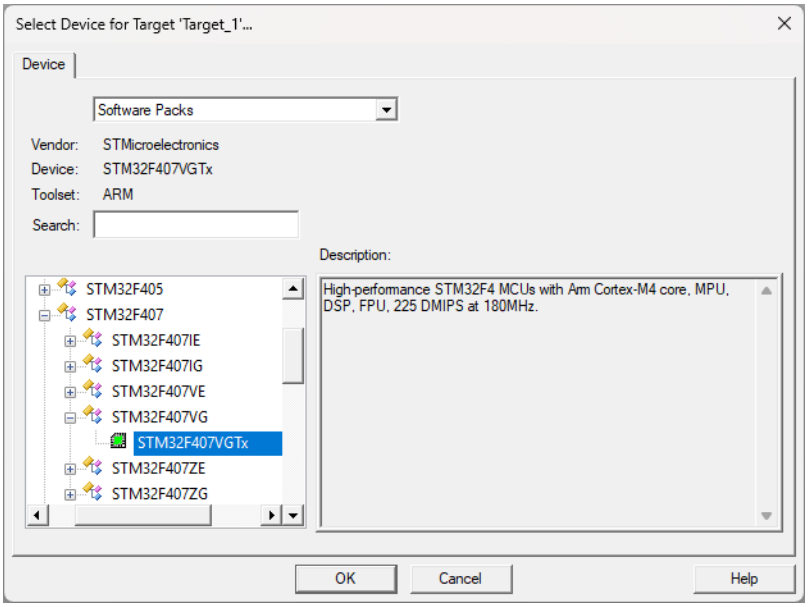


Figure 21 : Sélection de la référence STM32F407VGTx

3.2 Choix des composants logiciels (Run-Time Environment)

Keil ouvre la fenêtre **Manage Run-Time Environment**, qui permet d'ajouter au projet les composants logiciels nécessaires (CMSIS, pilotes, système temps réel…). Aucune case n'est cochée au départ.

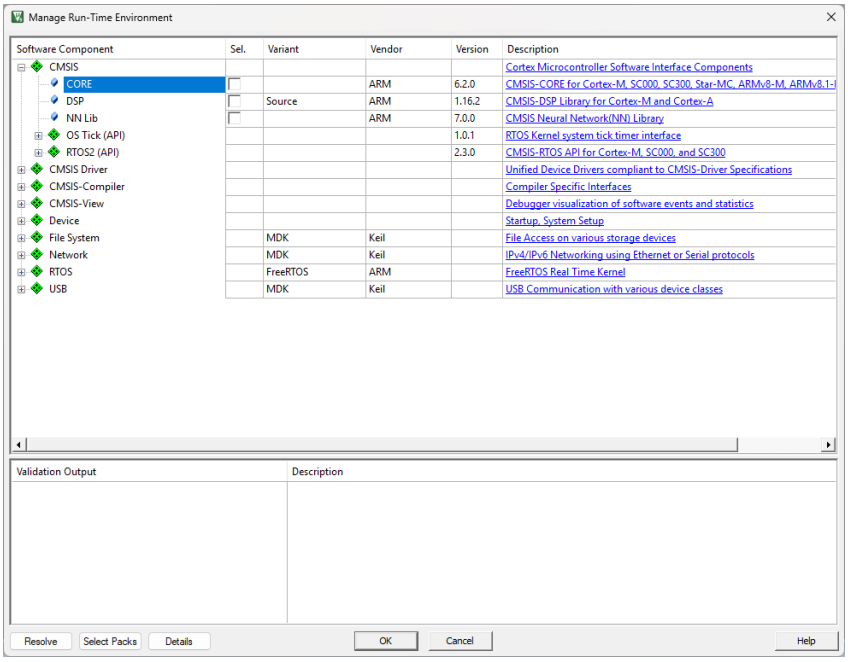


Figure 22 : Fenêtre Manage Run-Time Environment (avant sélection)

On coche **CMSIS > CORE**, qui apporte les fichiers de base du cœur Cortex-M, puis **Device > CubeMX** (« Configuration via STM32CubeMX »). La flèche verte à droite de CubeMX permet de lancer STM32CubeMX directement depuis Keil. Les zones vertes indiquent que les dépendances sont satisfaites. On valide avec **OK**.

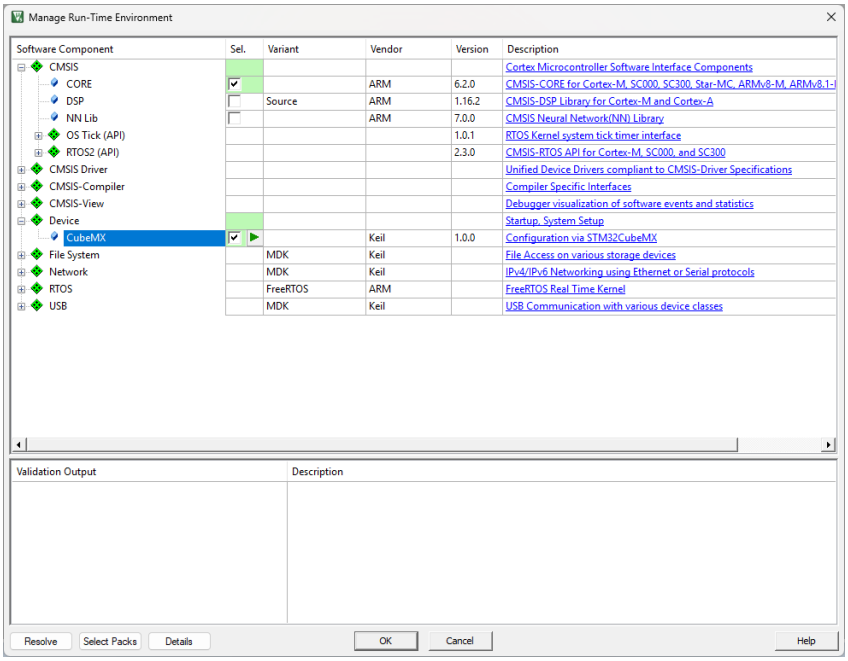


Figure 23 : Sélection de CMSIS > CORE et Device > CubeMX

Le projet **TP_LED** apparaît dans la fenêtre Project avec **Target_1**, **Source Group 1**, **CMSIS** et **Device**. La fenêtre Build Output indique que Keil lance STM32CubeMX. Le message d'erreur « Error #545 : Required input file from generator CubeMX is missing » est normal à ce stade, car le fichier de configuration n'existe pas encore : il disparaît après la première génération du code.

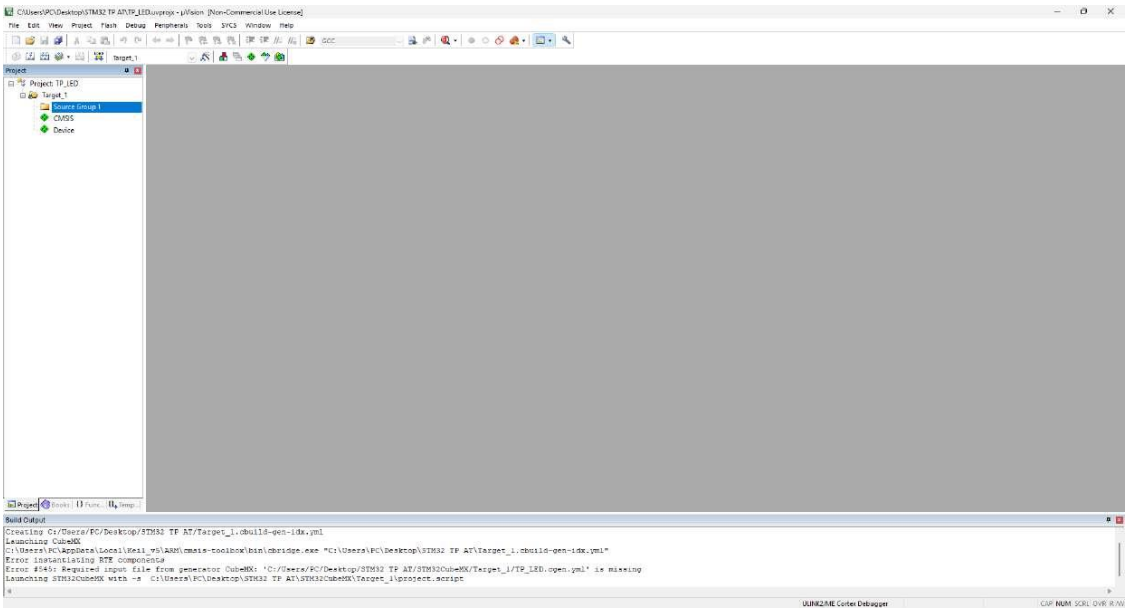


Figure 24 : Projet TP_LED dans Keil et lancement de STM32CubeMX

4. Configuration avec STM32CubeMX

4.1 Vue Pinout & Configuration

STM32CubeMX s'ouvre sur la vue **Pinout & Configuration** du **STM32F407VGTx** (boîtier LQFP100). L'indicateur « Unused GPIOs : 82/82 » montre qu'aucune broche n'est encore utilisée. Les broches en jaune sont les broches d'alimentation et de reset, imposées par le composant.

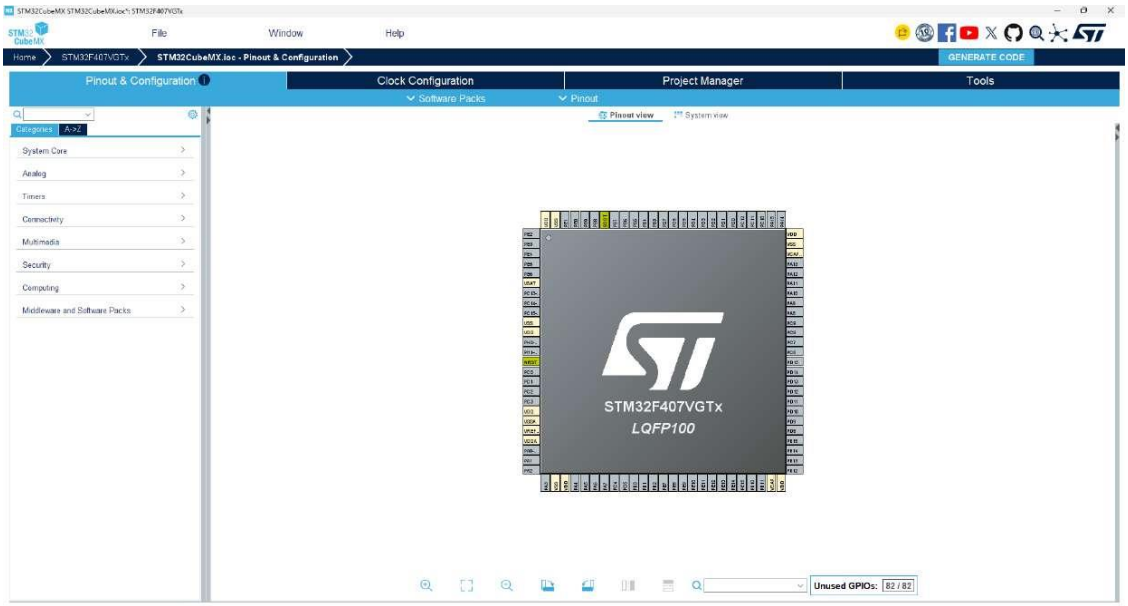


Figure 25 : Vue Pinout & Configuration du STM32F407VGTx

4.2 Configuration de la broche PD12 en sortie

On clique sur la broche **PD12** du boîtier : un menu contextuel liste toutes les fonctions possibles de la broche (FSMC_A17, TIM4_CH1, USART3_RTS, GPIO_Input, GPIO_Output, GPIO_Analog, GPIO_EXTI12…). On choisit **GPIO_Output**. La broche devient verte et le compteur passe à « 81/82 ». PD12 est la broche qui commande la LED verte LD4 de la carte.

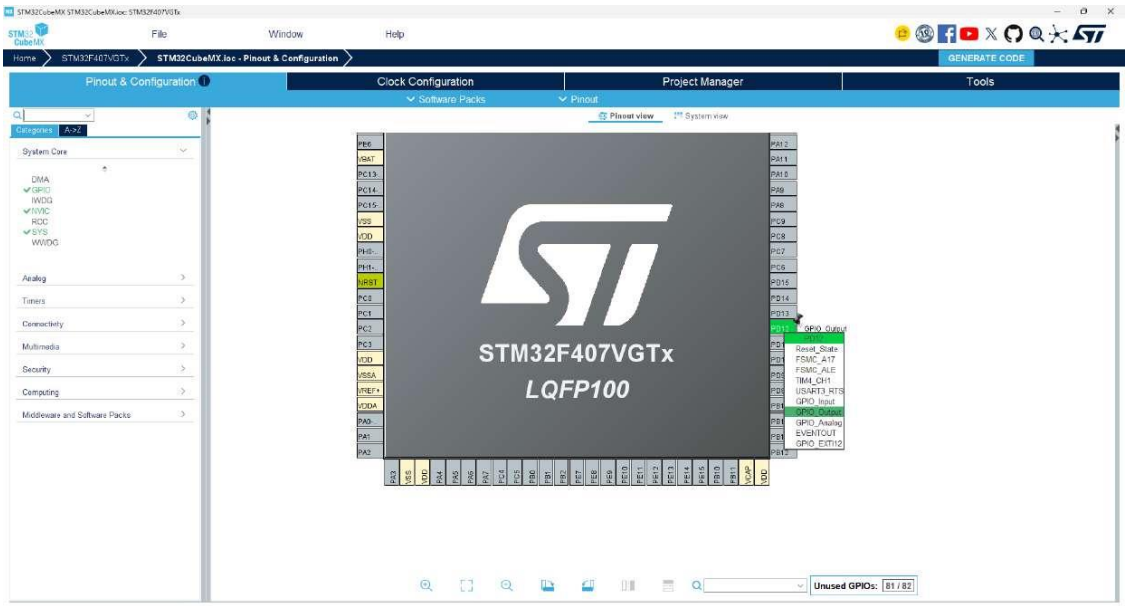


Figure 26 : Choix de GPIO_Output pour la broche PD12

4.3 Configuration de l'horloge

L'onglet **Clock Configuration** affiche l'arbre d'horloges du microcontrôleur. Dans notre configuration, l'oscillateur interne **HSI (16 MHz)** alimente le **PLL**, dont les diviseurs et multiplicateurs (÷8, ×100, ÷2) donnent une horloge système **SYSCLK = HCLK = 100 MHz**. Le bus APB1 fonctionne à 25 MHz (prédiviseur ÷4) et le bus APB2 à 50 MHz (prédiviseur ÷2). Aucune zone rouge n'apparaît : la configuration est valide.

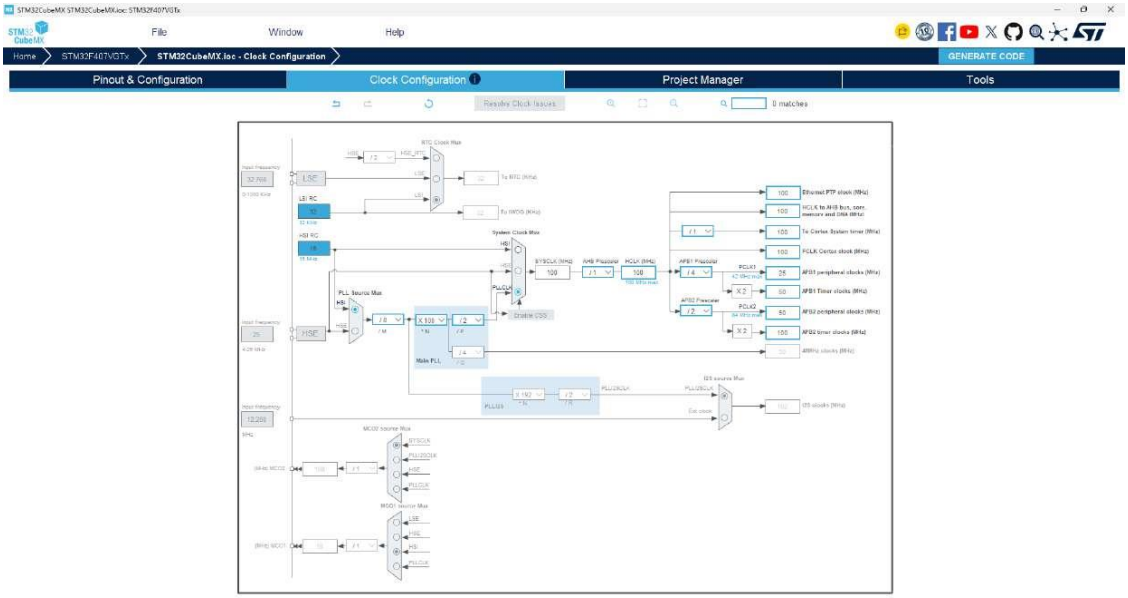


Figure 27 : Arbre d'horloges dans l'onglet Clock Configuration

4.4 Project Manager et génération du code

Dans l'onglet **Project Manager**, on renseigne le nom du projet (**STM32CubeMX**), son emplacement dans le dossier Target_1 du TP et la chaîne d'outils : **Toolchain / IDE : MDK-ARM** (Keil). La structure d'application est **Basic** ; la taille minimale du tas (heap) est 0x200 et celle de la pile (stack) 0x400. Le paquet firmware utilisé est STM32Cube FW_F4. On clique ensuite sur **GENERATE CODE**.

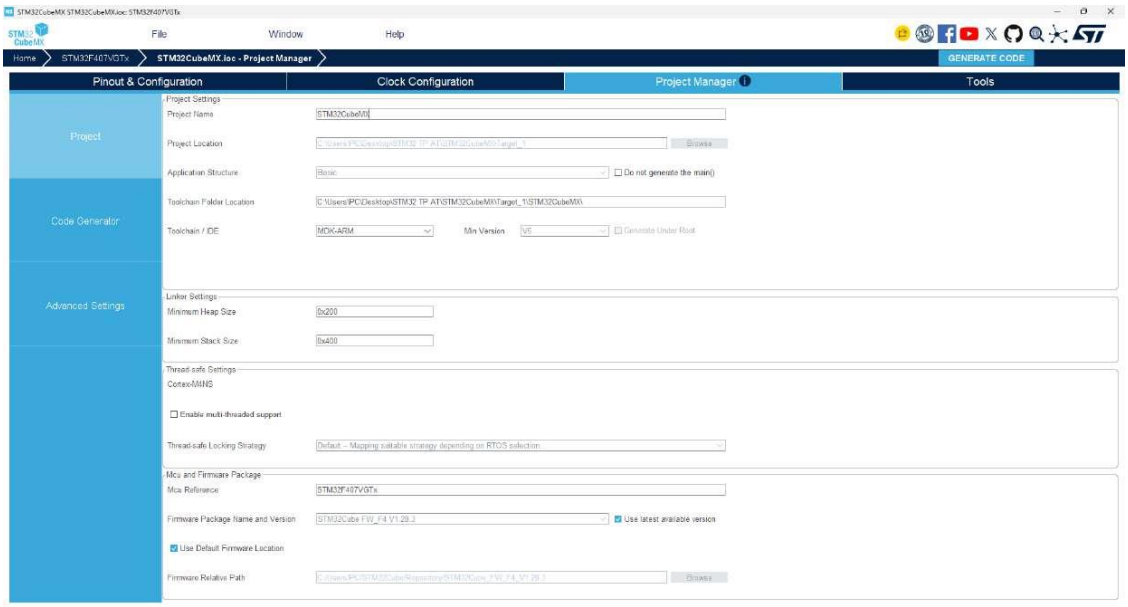


Figure 28 : Onglet Project Manager (MDK-ARM)

STM32CubeMX copie alors les fichiers des bibliothèques (HAL, CMSIS et fichiers de démarrage) dans le dossier du projet : la barre « Copying libraries files… » indique la progression.

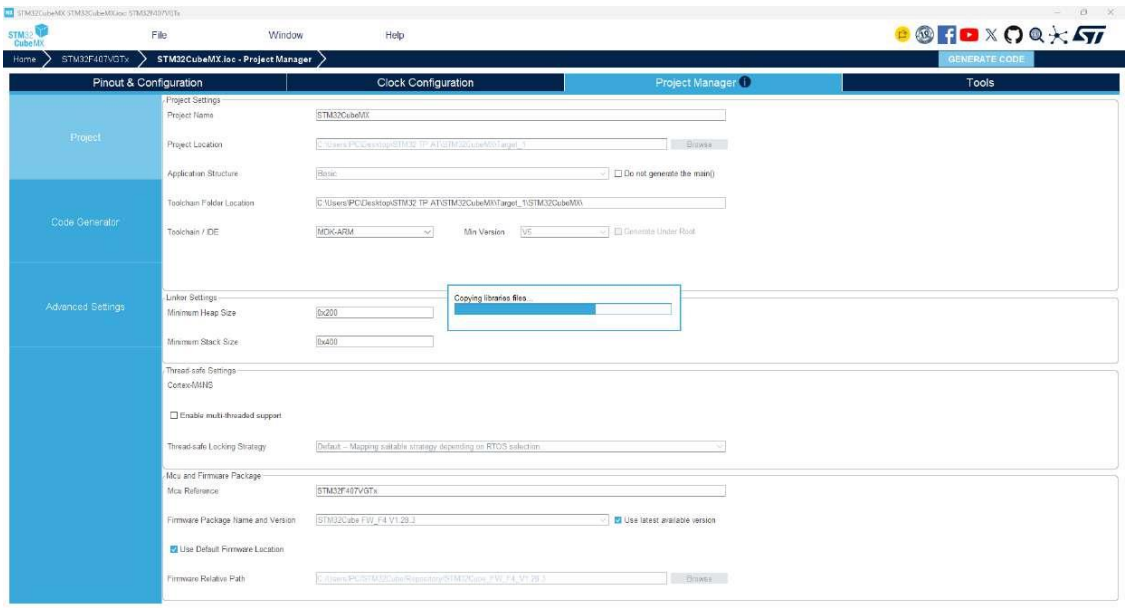


Figure 29 : Copie des fichiers de bibliothèques par STM32CubeMX

Une fois terminée, une fenêtre confirme « The Code is successfully generated » avec le chemin de génération et le langage du projet (C). On clique sur **Close** (ou **Open Project** pour revenir directement dans Keil).

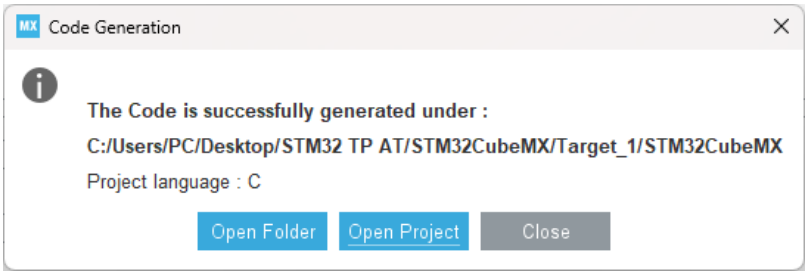


Figure 30: Confirmation de la génération du code

5. Ajout du fichier source et programmation

5.1 Création de notre propre fichier main.c

De retour dans Keil, le dossier **CubeMX::STM32CubeMX** contient les fichiers générés, dont un **main.c** de STM32CubeMX. Dans ce TP, nous écrivons notre propre programme par accès direct aux registres ; nous créons donc un nouveau fichier dans **Source Group 1**. Pour cela, on fait un clic droit sur **Source Group 1** et on choisit **Add New Item to Group 'Source Group 1'…**.

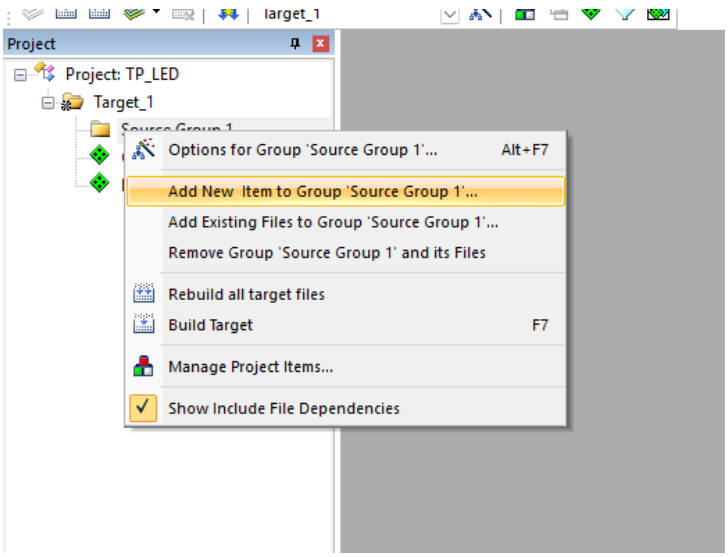


Figure 31 : Menu Add New Item to Group 'Source Group 1'

Dans la fenêtre **Add New Item**, on choisit **C File (.c)**, on saisit le nom **main** et on laisse l'emplacement du projet, puis on clique sur **Add**.

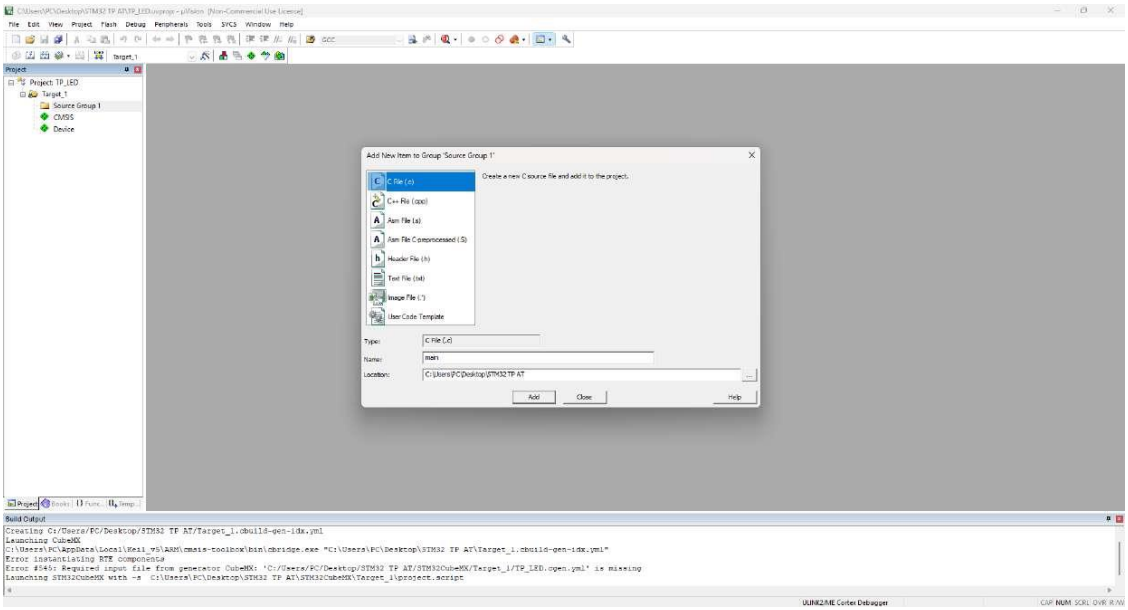


Figure 32 : Création du fichier main.c dans Source Group 1

5.2 Code du programme

On copie dans ce fichier le code suivant :

```
#include "stm32f407xx.h"

int main(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

/* Configure PA5 as output */
    GPIOA->MODER &= ~(3U <<(5 * 2));
    GPIOA->MODER |= (1U << (5 * 2));
    while (1){
        GPIOA->ODR ^= (1U << 5);
        for (volatile int i= 0; i < 10000; i++);
}
}
```

Explication du code :

- `#include "stm32f407xx.h"` : inclut le fichier d'en-tête qui définit les noms des registres du STM32F407 (`RCC`, `GPIOA`…) et de leurs bits.
- `RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;` : active l'horloge du port GPIOA. Sans horloge, un port GPIO ne réagit pas aux écritures dans ses registres. Le OU logique (`|=`) met ce bit à 1 sans modifier les autres.
- `GPIOA->MODER &= ~(3U << (5 * 2));` : le registre MODER utilise 2 bits par broche ; pour la broche 5 ce sont les bits 10 et 11. Cette ligne les met d'abord à 00 (masque `3U` = binaire 11, décalé de 10 positions, puis inversé).
- `GPIOA->MODER |= (1U << (5 * 2));` : met ces deux bits à 01, ce qui configure la broche en **sortie**.
- `while (1)` : boucle infinie, indispensable dans un programme embarqué qui ne se termine jamais.
- `GPIOA->ODR ^= (1U << 5);` : le OU exclusif (`^=`) **inverse** le bit 5 du registre de sortie ODR : s'il valait 0 il passe à 1 (LED allumée), s'il valait 1 il passe à 0 (LED éteinte).
- `for (volatile int i = 0; i < 10000; i++);` : boucle de temporisation. Le mot-clé `volatile` empêche le compilateur de supprimer cette boucle qui ne fait « rien ». Chaque tour prend un peu de temps, ce qui fixe la durée entre deux inversions de la LED.

5.3 Exclusion du main.c généré par STM32CubeMX

Le projet contient désormais deux fichiers **main.c** : le nôtre (dans Source Group 1) et celui généré par STM32CubeMX. Comme un programme ne peut avoir qu'une seule fonction `main()`, la compilation échouerait avec une erreur de définition multiple. Il faut donc **exclure de la compilation** le main.c de STM32CubeMX. La capture suivante montre l'arborescence du projet avec notre code et une compilation réussie : **0 Error(s), 0 Warning(s)**.

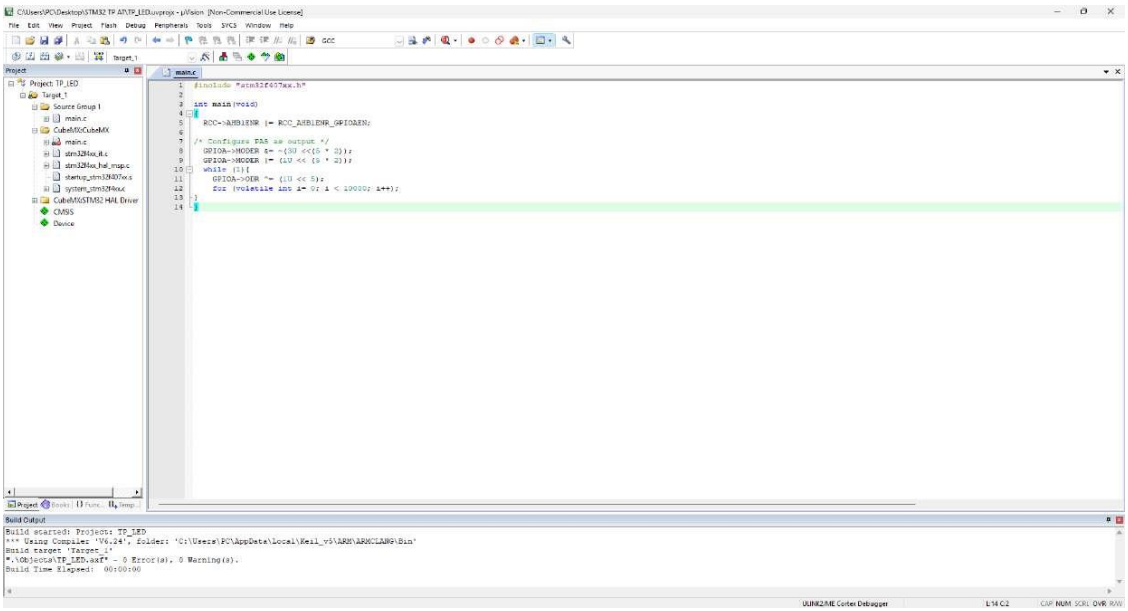


Figure 33 : Code du TP dans main.c et compilation sans erreur

Pour exclure le fichier, on fait un clic droit sur le **main.c** du groupe CubeMX::STM32CubeMX et on choisit **Options for File 'main.c'…** (raccourci Alt+F7).

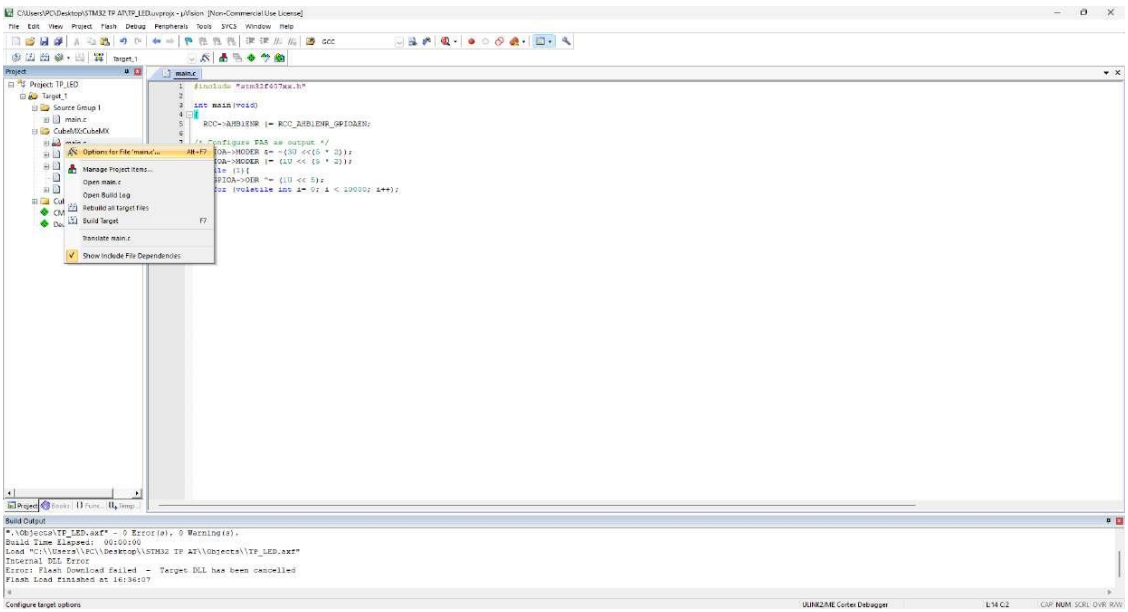


Figure 34 : Menu Options for File 'main.c' sur le main.c de STM32CubeMX

Dans l'onglet **Properties**, on **décoche « Include in Target Build »**. Le chemin affiché (…\STM32CubeMX\Target_1\STM32CubeMX\Src\main.c) confirme qu'il s'agit bien du fichier généré par STM32CubeMX. On valide avec **OK** : un seul `main()` est maintenant compilé.

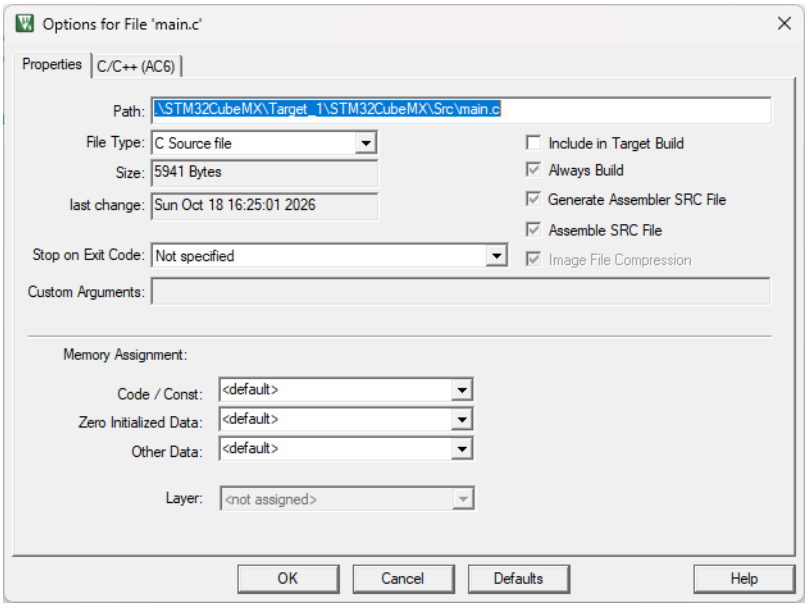


Figure 35 : Exclusion du main.c généré : case Include in Target Build décochée

5.4 Remarque importante : choix de la broche

Le code ci-dessus configure la broche **PA5** (port GPIOA). Or, dans STM32CubeMX, nous avons configuré la broche **PD12**, qui commande la LED verte LD4 de la carte STM32F4DISCOVERY. Sur cette carte, PA5 est reliée à l'accéléromètre (ligne d'horloge SPI1) et ne commande aucune LED ; la broche PA5 est celle de la LED sur la carte Nucleo-F4. Pour que la LED de la carte Discovery clignote avec la configuration PD12 de STM32CubeMX, il faut commander le port **D** et la broche **12** :

```
#include "stm32f407xx.h"

int main(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIODEN;      /* horloge du port D */

    /* PD12 en sortie (LED verte LD4) */
    GPIOD->MODER &= ~(3U << (12 * 2));
    GPIOD->MODER |=  (1U << (12 * 2));

    while (1) {
        GPIOD->ODR ^= (1U << 12);              /* inverse PD12 */
        for (volatile int i = 0; i < 1000000; i++);  /* temporisation visible */
    }
}
```

Dans cette variante, on active l'horloge du port D (`GPIODEN`), on configure les bits 24 et 25 de `GPIOD->MODER` (broche 12 × 2) et on inverse le bit 12 de `GPIOD->ODR`. La valeur de la temporisation est aussi augmentée : à 10 000 tours, la LED s'inverse plusieurs centaines de fois par seconde et semble simplement allumée en permanence ; à 1 000 000 de tours, le clignotement devient visible à l'œil.

6. Téléchargement du programme dans la carte

Après la compilation, il reste à transférer le programme dans la mémoire Flash du microcontrôleur en passant par le ST-LINK.

1. Brancher la carte au PC avec le câble USB sur le connecteur mini-USB du ST-LINK. La LED LD2 s'allume (alimentation) et LD1 signale la communication ST-LINK.
2. Ouvrir **Options for Target** (icône en forme de baguette magique ou **Alt+F7**).

3. Dans l'onglet **Debug**, choisir à droite **ST-Link Debugger** dans la liste « Use » (par défaut, Keil propose un autre débogueur, ce qui provoque une erreur lors du téléchargement).
4. Cliquer sur **Settings** : dans l'onglet **Debug**, le port doit être **SW** et la zone « SW Device » doit afficher le cœur ARM du STM32F407, ce qui prouve que la carte est bien détectée.
5. Dans l'onglet **Flash Download** (ou **Utilities**), vérifier que l'algorithme **STM32F4xx Flash** est présent et cocher **Reset and Run**. Valider avec **OK**.
6. Compiler le projet (**F7**) puis lancer le téléchargement avec **Flash > Download** (**F8**). Le Build Output affiche la progression, puis « Programming Done » et « Verify OK ».

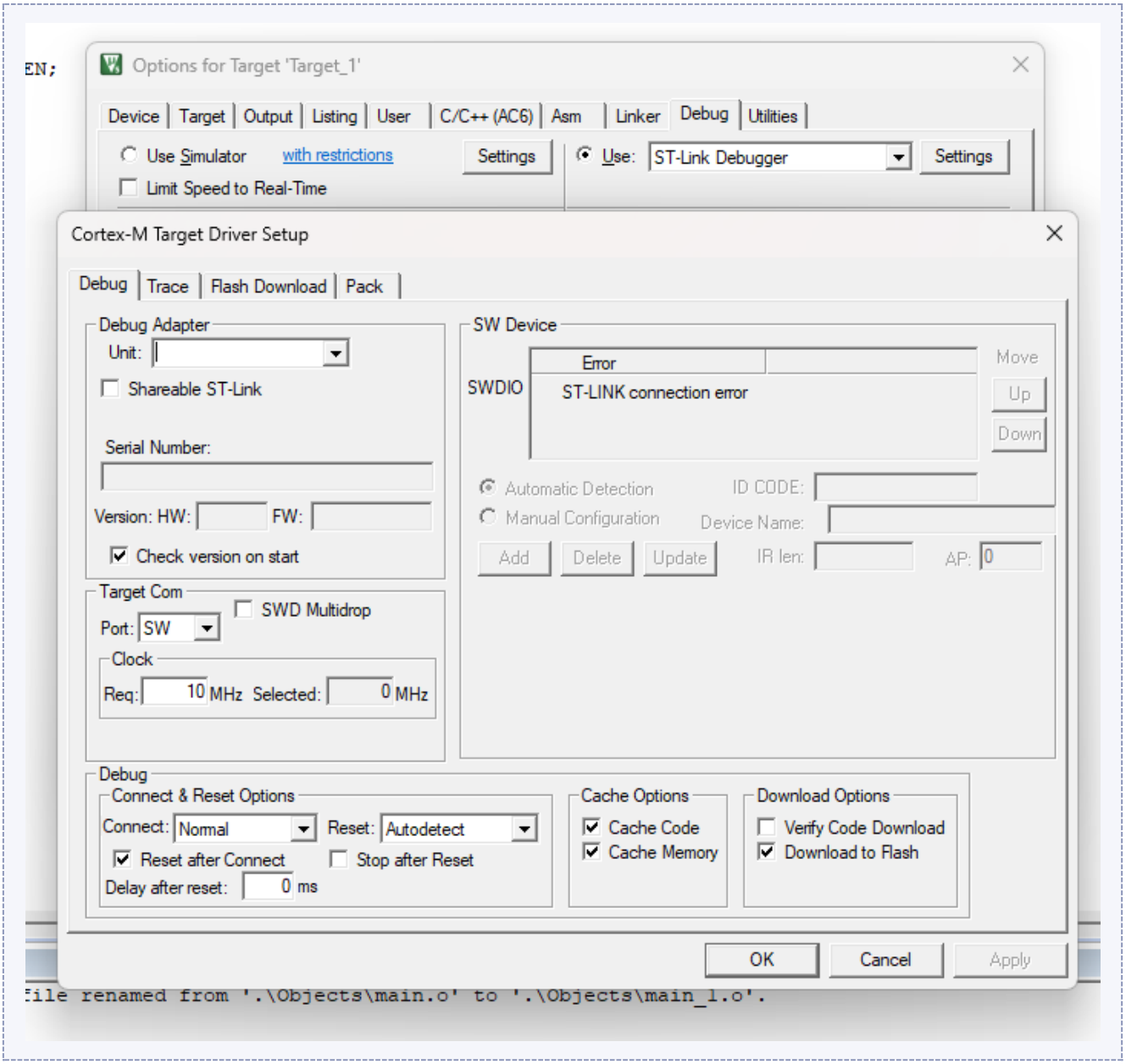


Figure 36 : Sélection de ST-Link Debugger dans Options for Target > Debug

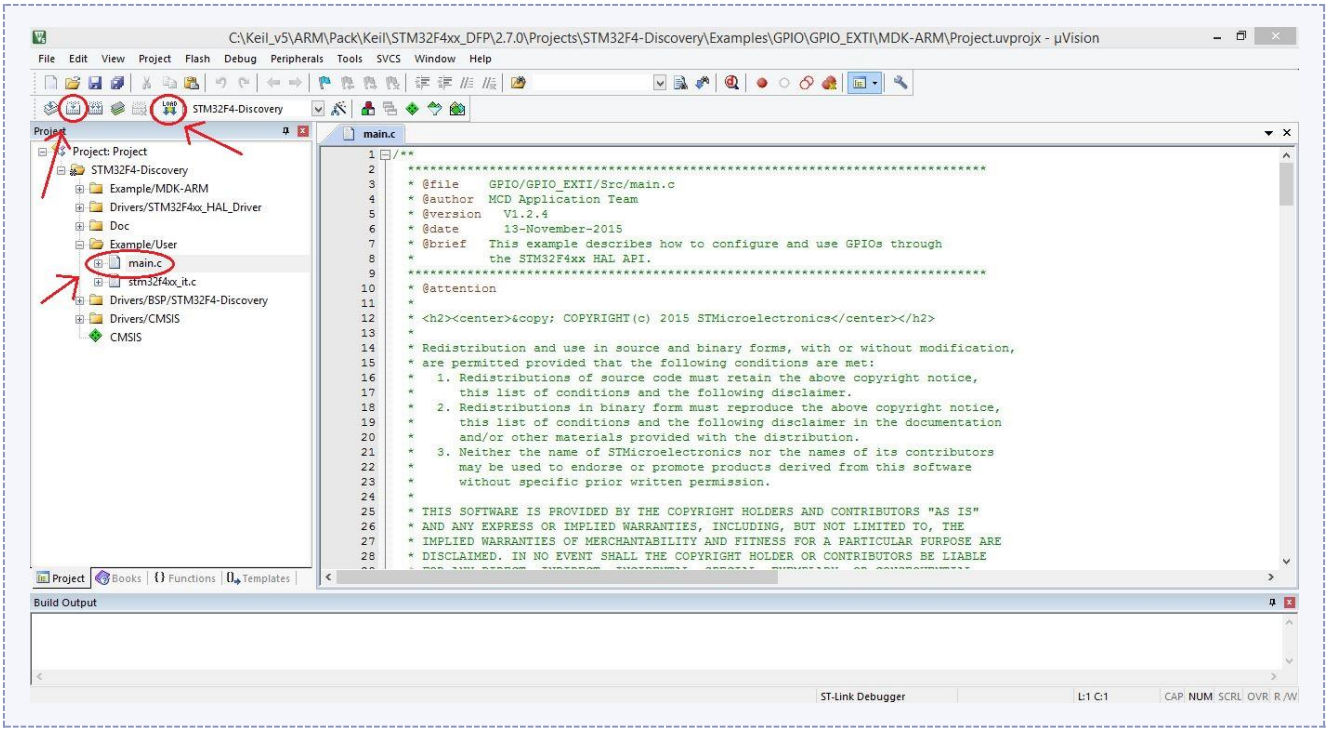


Figure 37 : Réglages ST-Link (Settings) et algorithme de programmation Flash

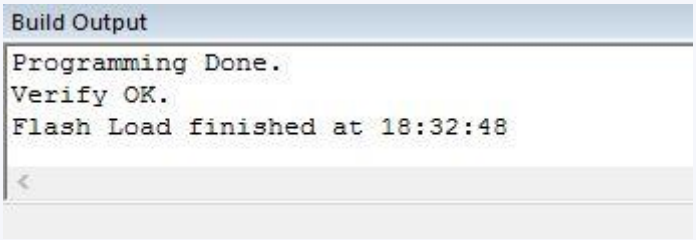


Figure 38 : Téléchargement du programme dans la Flash (Flash > Download)

7. Résultats obtenus

Après le téléchargement, le programme s'exécute dans le microcontrôleur : la broche configurée est inversée en boucle et la LED correspondante de la carte clignote. Ce résultat valide toute la chaîne de développement : création du projet, configuration par STM32CubeMX, programmation en C par accès aux registres, compilation et téléchargement par le ST-LINK.

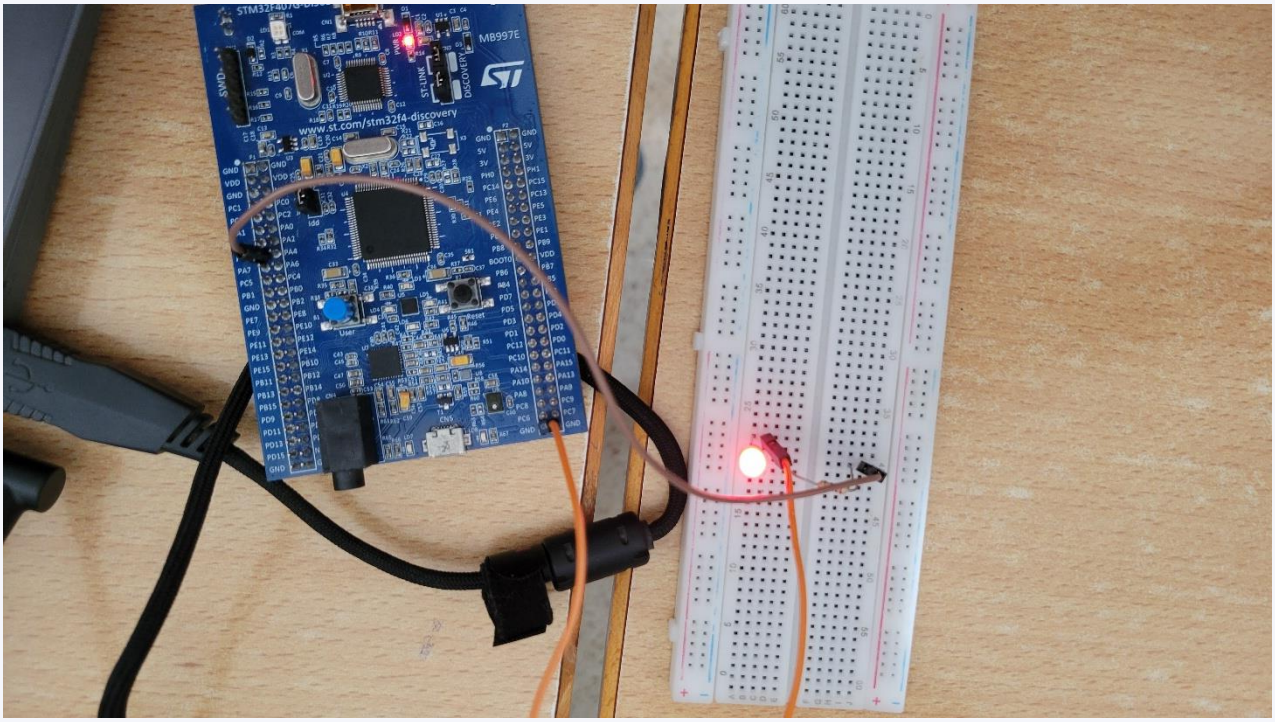


Figure 39 : Résultat : LED de la carte STM32F407 qui clignote

8. Exercice 2 : clignotement d'une LED externe (broche PA5)

8.1 Objectif et matériel

Après avoir fait clignoter la LED intégrée à la carte (LD4 sur **PD12**), on commande cette fois une **LED externe** montée sur une breadboard et reliée à la broche **PA5** du port A. Cet exercice montre qu'une broche GPIO peut piloter un composant extérieur à la carte, et il utilise une fonction de temporisation `delay()` réutilisable.

Tableau 13 : Matériel de l'exercice 2 et câblage

Élément	Détail
Carte	STM32F407 (STM32F4DISCOVERY), alimentée et programmée par le câble USB du ST-LINK
Composants	1 LED rouge, 1 résistance de limitation (220 Ω recommandée), 1 breadboard, fils de liaison
Câblage	Broche PA5 de la carte vers l'anode de la LED (patte longue, via la résistance) ; cathode de la LED (patte courte) vers une broche GND de la carte
Logiciels	Keil µVision 5, STM32CubeMX, pilote ST-LINK

8.2 Étapes de réalisation

La démarche est la même que celle de l'exercice 1 (sections 3 à 6) ; seule la broche configurée change. Les étapes sont :

1. **Créer le projet Keil** : Project > New µVision Project, choisir le dossier et le nom du projet, puis le composant STM32F407VGTx (figures 16 à 21).
2. **Ajouter les composants logiciels** : dans Manage Run-Time Environment, cocher CMSIS > CORE et Device > CubeMX (figures 22 et 23), puis lancer STM32CubeMX avec la flèche verte.

3. **Configurer la broche** : dans Pinout & Configuration, cliquer sur **PA5** et choisir **GPIO_Output** (même manipulation que la figure 26, avec PA5 à la place de PD12).
4. **Régler l'horloge** : onglet Clock Configuration, HSI vers PLL, SYSCLK = 100 MHz (figure 27).
5. **Générer le code** : onglet Project Manager, Toolchain / IDE = MDK-ARM, puis GENERATE CODE (figures 28 à 30).
6. **Ajouter notre fichier main.c** : clic droit sur Source Group 1 > Add New Item to Group > C File (.c), nom main (figures 31 et 32).
7. **Exclure le main.c de STM32CubeMX** : Options for File (Alt+F7) puis décocher Include in Target Build (figures 34 et 35).
8. **Compiler** avec F7 : le Build Output doit afficher 0 Error(s), 0 Warning(s).
9. **Télécharger** dans la carte avec Flash > Download (F8) après avoir choisi ST-Link Debugger (section 10).

8.3 Code du programme

Le fichier main.c ajouté dans Source Group 1 contient le code suivant :

```
#include "stm32f407xx.h"

static void delay(volatile uint32_t time)
{
    while (time--)
    {
        __NOP();
    }
}

int main(void)
{
    /* Horloge du port A */
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

    /* PA5 en sortie */
    GPIOA->MODER &= ~(3U << (5 * 2));
    GPIOA->MODER |=  (1U << (5 * 2));

    while (1)
    {
        GPIOA->ODR |=  (1U << 5);   /* LED allumée */
        delay(1000000);
        GPIOA->ODR &= ~(1U << 5);   /* LED éteinte */
        delay(1000000);
    }
}
```

Explication du code :

- `static void delay(volatile uint32_t time)` : fonction de temporisation. Elle décrémente le compteur jusqu'à 0 en exécutant l'instruction `__NOP()` (« ne rien faire ») à chaque tour. Le mot-clé `volatile` empêche le compilateur de supprimer la boucle. Le paramètre fixe la durée : avec 1 000 000, l'attente est de l'ordre de quelques dixièmes de seconde, donc visible à l'œil.
- `RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;` : active l'horloge du port A.
- `GPIOA->MODER &= ~(3U << (5 * 2));` puis `GPIOA->MODER |= (1U << (5 * 2));` : mettent les bits 10 et 11 du registre MODER à 01, c'est-à-dire la broche PA5 en sortie.

- `GPIOA->ODR |= (1U << 5);` : met le bit 5 du registre de sortie à 1, la broche passe à 3,3 V et la LED s'allume.
- `GPIOA->ODR &= ~(1U << 5);` : remet le bit 5 à 0, la broche passe à 0 V et la LED s'éteint.

> **Différence avec l'exercice 1 :** ici on écrit explicitement l'état allumé puis l'état éteint (opérations |= puis &= ~), alors que dans l'exercice 1 on inversait le bit avec le OU exclusif (^=). Les deux méthodes donnent un clignotement ; la méthode explicite est préférable quand on commande plusieurs broches, comme dans l'exercice 3.

8.4 Résultat

Après le téléchargement, la LED externe s'allume puis s'éteint en boucle avec une période d'environ une demi-seconde par état, ce qui valide le câblage et le programme.

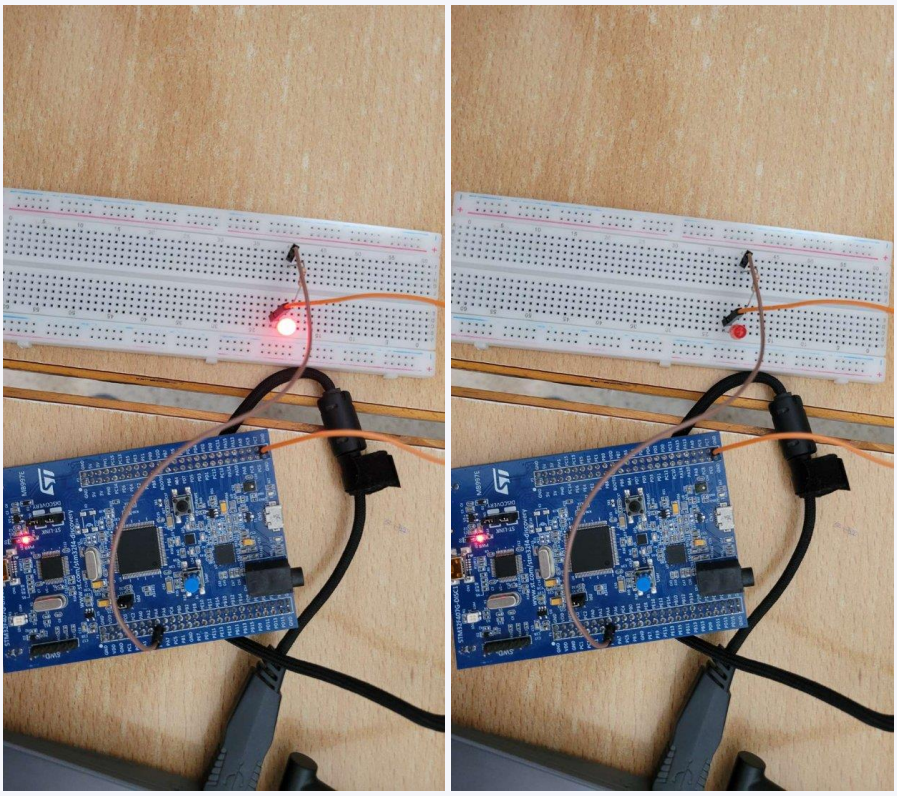


Figure 40 : LED externe reliée à PA5 : (a) allumée, (b) éteinte

9. Exercice 3 : feu tricolore (pare-feux)

9.1 Objectif et câblage

Cet exercice simule un **feu de circulation** avec trois LED (verte, jaune, rouge) commandées par trois broches du port A. Le feu suit en boucle la séquence : **vert, jaune, rouge, jaune**. Chaque LED est branchée en série avec une résistance de limitation ; les cathodes sont reliées à la masse (GND) de la carte.

Tableau 14 : Affectation des broches du feu tricolore

Couleur	Broche	Rôle dans le programme
Verte	PA5	Feu vert : passage autorisé
Jaune	PA4	Feu jaune : attention, changement d'état
Rouge	PA7	Feu rouge : arrêt

9.2 Création du projet et choix des broches

On crée d'abord un nouveau projet Keil (nom **parefeux**, dossier TP2_PARE_FEUX) pour le composant STM32F407VGTx, avec les composants CMSIS > CORE et Device > CubeMX, comme aux sections 3.1 et 3.2. On lance ensuite STM32CubeMX depuis Keil.

Dans la vue **Pinout & Configuration**, on clique successivement sur **PA4**, **PA5** et **PA7** et on choisit **GPIO_Output** pour chacune. Les trois broches deviennent vertes et le compteur « Unused GPIOs » passe à 79/82.

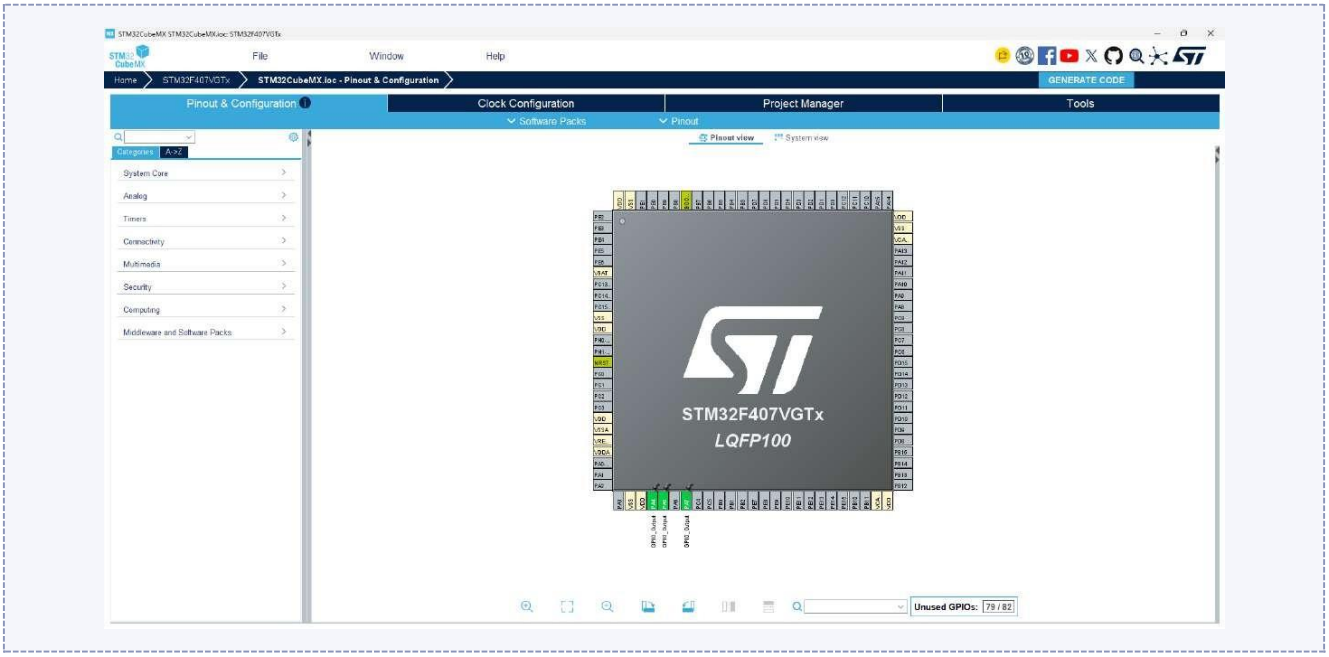


Figure 41 : Broches PA4, PA5 et PA7 configurées en GPIO_Output

9.3 Configuration de l'horloge

Dans l'onglet **Clock Configuration**, on conserve la même configuration que dans l'exercice 1 : l'oscillateur interne HSI (16 MHz) alimente le PLL, ce qui donne SYSCLK = HCLK = 100 MHz. Aucune zone rouge n'apparaît.

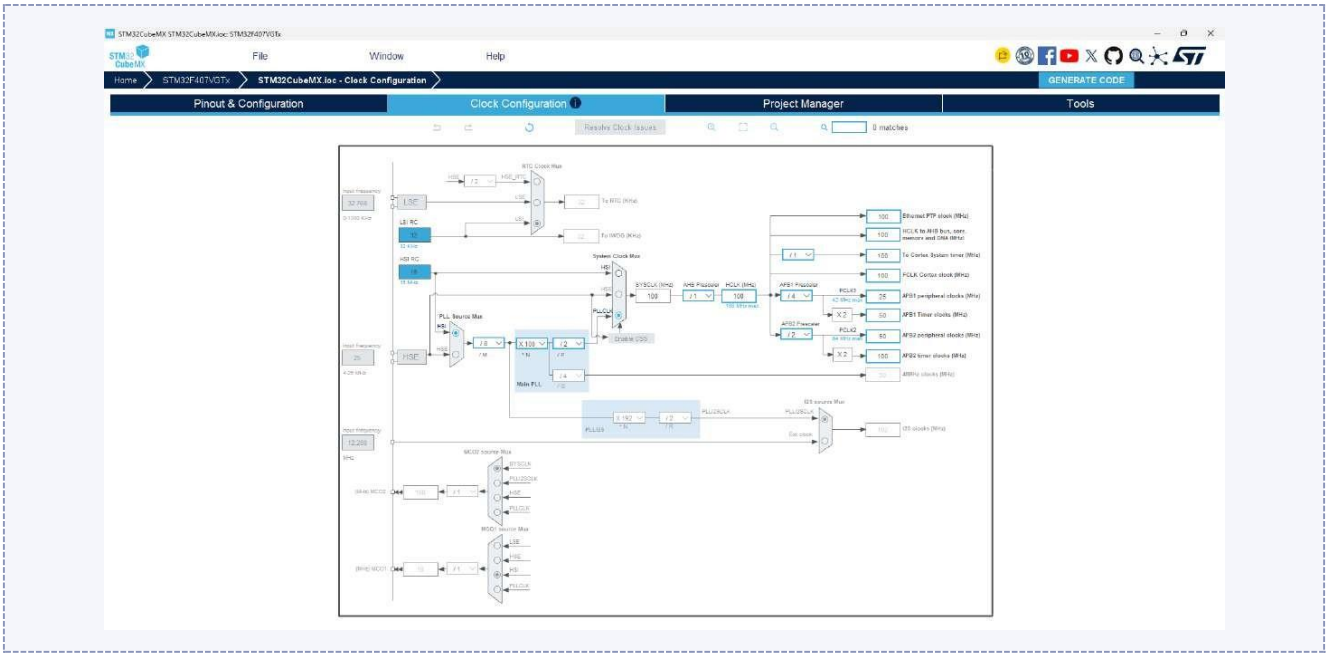


Figure 42 : Arbre d'horloges (Clock Configuration)

Remarque : comme le main.c généré par STM32CubeMX est exclu de la compilation, la fonction qui applique ces réglages d'horloge n'est pas exécutée. Notre programme tourne donc sur l'horloge par défaut du microcontrôleur (HSI, 16 MHz). Cela n'empêche pas le fonctionnement, mais la durée de delay() dépend de cette fréquence.

9.4 Génération du code

Dans l'onglet **Project Manager**, on renseigne le nom et l'emplacement du projet, puis **Toolchain / IDE : MDK-ARM**. Un clic sur **GENERATE CODE** lance la génération : la fenêtre « Generating user source code » indique la progression.

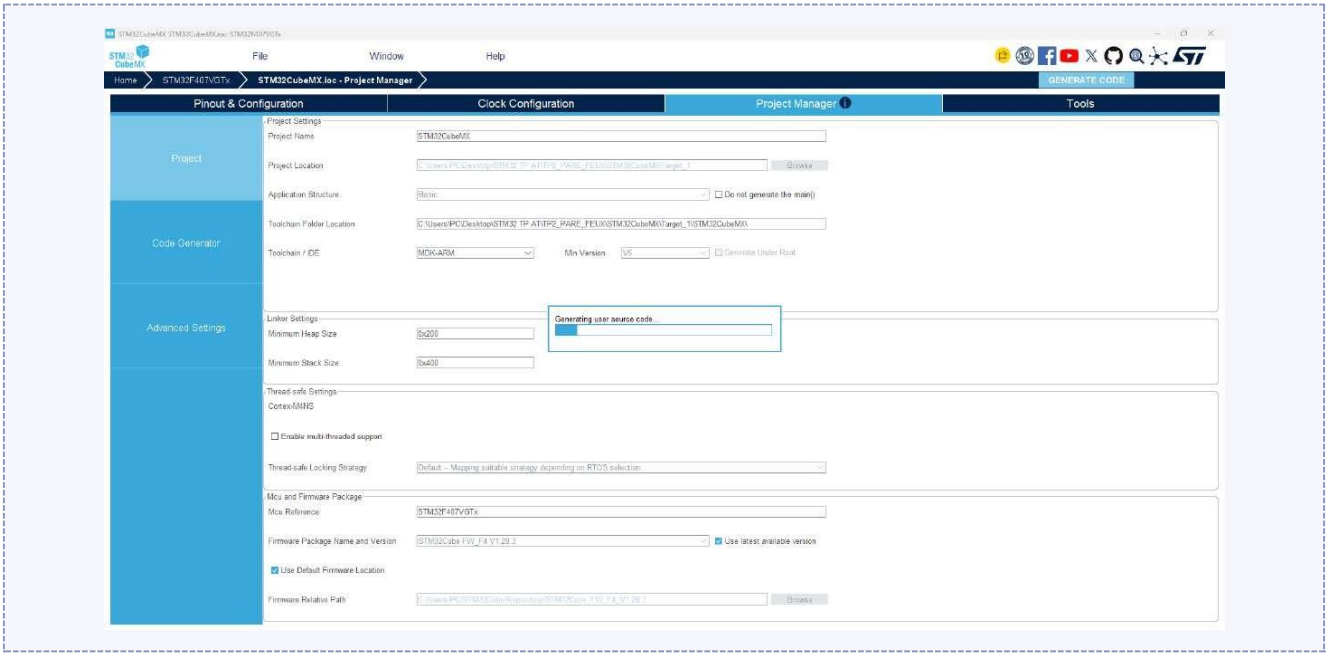


Figure 43 : Project Manager et génération du code

9.5 Ajout du fichier main.c dans Source Group 1

De retour dans Keil, on ajoute notre fichier source :

1. Clic droit sur **Source Group 1** puis **Add New Item to Group 'Source Group 1'…**.
2. Choisir **C File (.c)**, saisir le nom **main** puis cliquer sur **Add**.
3. Coller le code du paragraphe 9.6 dans ce fichier.
4. Exclure le main.c généré par STM32CubeMX : clic droit dessus, **Options for File…** (Alt+F7), décocher **Include in Target Build**.
5. Compiler avec **F7**.

9.6 Code du programme

```
#include "stm32f407xx.h"

static void delay(volatile uint32_t time)
{
    while (time--)
    {
        __NOP();
    }
}

int main(void)
{
    /* Activation de l'horloge de GPIOA */
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
```

```
    /* Configuration des broches PA4, PA5 et PA7 en sortie */
    GPIOA->MODER &= ~((3U << (5 * 2)) |
                      (3U << (4 * 2)) |
                      (3U << (7 * 2)));

    GPIOA->MODER |=  ((1U << (5 * 2)) |
                      (1U << (4 * 2)) |
                      (1U << (7 * 2)));

    while (1)
    {
        /* Feu vert */
        GPIOA->ODR |=  (1U << 5);
        GPIOA->ODR &= ~((1U << 4) | (1U << 7));
        delay(1000000);

        /* Feu jaune */
        GPIOA->ODR &= ~(1U << 5);
        GPIOA->ODR |=  (1U << 4);
        GPIOA->ODR &= ~(1U << 7);
        delay(1000000);

        /* Feu rouge */
        GPIOA->ODR &= ~((1U << 5) | (1U << 4));
        GPIOA->ODR |=  (1U << 7);
        delay(1000000);

        /* Feu jaune */
        GPIOA->ODR &= ~(1U << 5);
        GPIOA->ODR |=  (1U << 4);
        GPIOA->ODR &= ~(1U << 7);
        delay(1000000);
    }
}
```

Explication du code :

- La fonction `delay()` et l'activation de l'horloge de GPIOA sont identiques à l'exercice 2.
- Les deux instructions sur `MODER` travaillent sur trois broches à la fois : la première met à 00 les paires de bits de PA4, PA5 et PA7 (bits 8-9, 10-11 et 14-15), la seconde les met à 01 (sortie). Les masques sont regroupés avec l'opérateur OU (|).
- Dans la boucle infinie, chaque étape allume une seule LED et éteint les deux autres : on utilise `|=` pour mettre un bit à 1 et `&= ~` pour le mettre à 0, sans toucher aux autres broches du port.
- L'appel `delay(1000000)` maintient chaque état pendant une durée visible avant de passer au suivant.

Tableau 15 : Séquence du feu tricolore (1 = broche à 3,3 V, LED allumée)

Étape	PA5 (vert)	PA4 (jaune)	PA7 (rouge)	Durée
Feu vert	1	0	0	delay
Feu jaune	0	1	0	delay
Feu rouge	0	0	1	delay
Feu jaune	0	1	0	delay

Correction du commentaire : dans le fichier d'origine, le commentaire annonçait « PA5, PA6 et PA7 » alors que le code configure PA4, PA5 et PA7 ; le commentaire a été corrigé ci-dessus. Le code, lui, était correct et correspond bien au câblage.

9.7 Compilation

La capture suivante montre le projet **parefeux** avec le code dans main.c. Le Build Output indique une compilation sans erreur : **0 Error(s), 0 Warning(s)**.

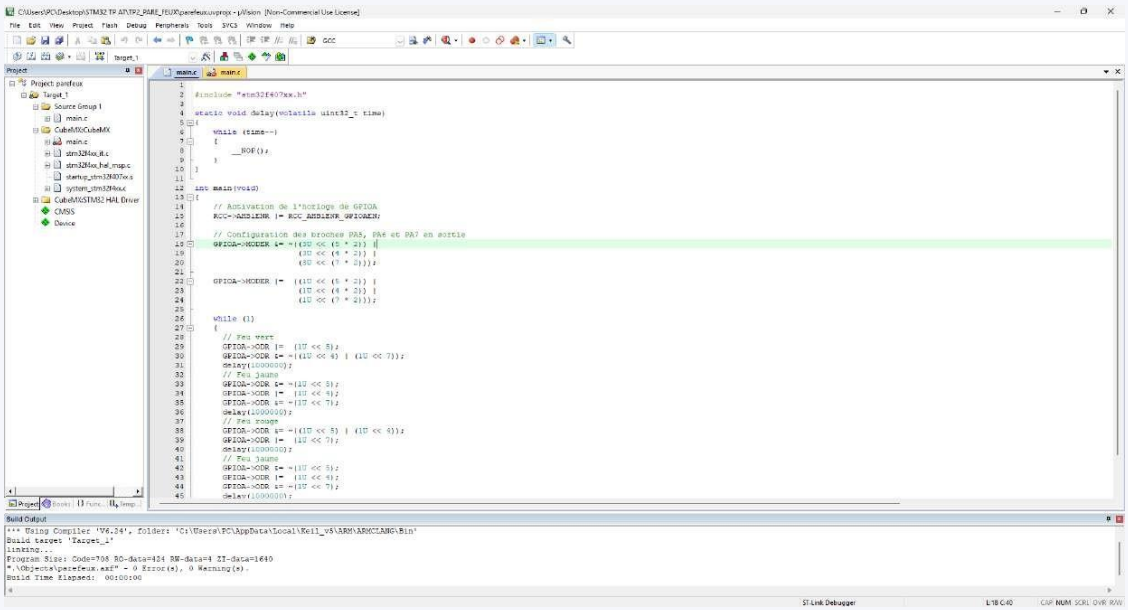


Figure 44 : Code du feu tricolore dans Keil et compilation sans erreur

9.8 Téléchargement et résultat

Le programme est téléchargé dans la carte avec **Flash > Download** (voir la section 10 pour la configuration du ST-Link) ; le Build Output affiche « Programming Done » et « Verify OK » (figure 38). Le feu fonctionne ensuite en boucle : la LED verte s'allume, puis la jaune, puis la rouge, puis de nouveau la jaune.

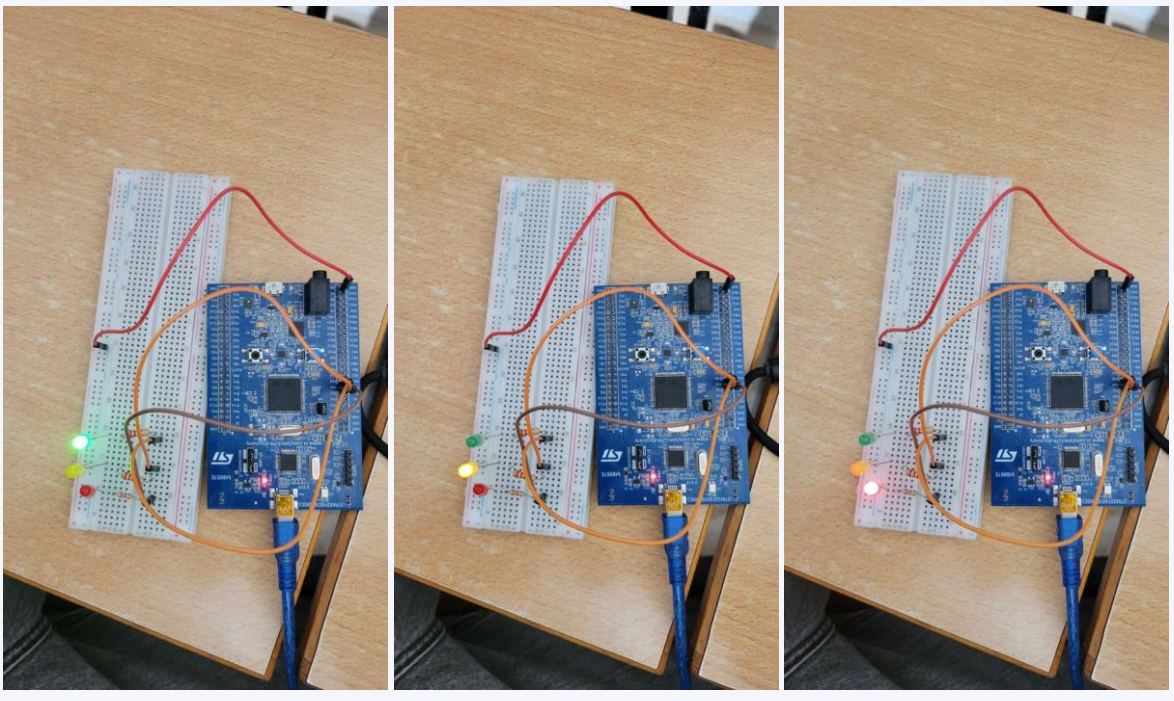


Figure 45 : Feu tricolore : (a) feu vert, (b) feu jaune, (c) feu rouge

10. Configuration du ST-Link Debugger dans Keil µVision 5

10.1 Rôle du ST-Link

Le ST-Link intégré à la carte relie le PC au microcontrôleur par le protocole **SWD** (Serial Wire Debug). C'est lui qui écrit le programme dans la mémoire Flash et qui permet de le déboguer. Si Keil n'est pas réglé pour l'utiliser, le téléchargement échoue (message « Internal DLL Error » ou « Flash Download failed »). La configuration est à refaire pour chaque nouveau projet.

10.2 Vérification de la connexion

On branche la carte au PC par le câble USB, puis on ouvre le Gestionnaire de périphériques de Windows : l'entrée **ST-Link Debug** doit apparaître sous USB Composite Device, sans point d'exclamation jaune. Dans le cas contraire, il faut installer le pilote STSW-LINK009 (section 5 du chapitre 3).

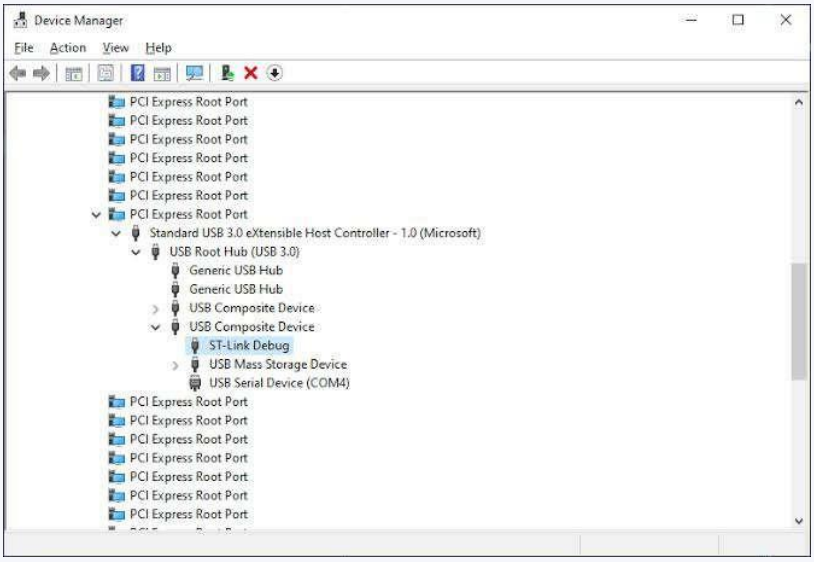


Figure 46 : ST-Link Debug reconnu dans le Gestionnaire de périphériques

10.3 Réglages dans Options for Target

On ouvre **Options for Target** (icône en forme de baguette magique dans la barre d'outils, ou raccourci **Alt+F7**) et on suit les étapes ci-dessous.

1. **Onglet Debug** : dans la partie droite de la fenêtre, choisir **ST-Link Debugger** dans la liste « Use ». La partie gauche (« Use Simulator ») doit rester décochée.
2. **Bouton Settings** : dans l'onglet Debug, vérifier que le port est **SW** (Serial Wire). La zone « SW Device » doit afficher le cœur ARM du STM32F407 : c'est la preuve que la carte est détectée.
3. **Onglet Flash Download** (dans la même fenêtre Settings) : vérifier la présence de l'algorithme **STM32F4xx Flash** (1 Mo, adresse de départ 0x08000000), choisir **Erase Sectors**, cocher **Program**, **Verify** et **Reset and Run**. Valider avec OK.
4. **Onglet Utilities** (Options for Target) : cocher « Use Target Driver for Flash Programming » avec **ST-Link Debugger** dans la liste, puis valider avec OK.
5. **Compiler** avec l'icône Build (F7).
6. **Télécharger** avec l'icône Download (F8) ou le menu Flash > Download. Le Build Output affiche « Programming Done », « Verify OK » puis l'heure de fin du chargement (figure 38).

La figure suivante montre la barre d'outils de Keil : l'icône entourée à gauche est **Build**, l'icône entourée à droite (LOAD) est **Download**, et le nom du débogueur choisi (ST-Link Debugger) s'affiche dans la barre d'état en bas à droite. La capture provient du projet exemple GPIO_EXTI fourni avec le pack STM32F4xx_DFP.

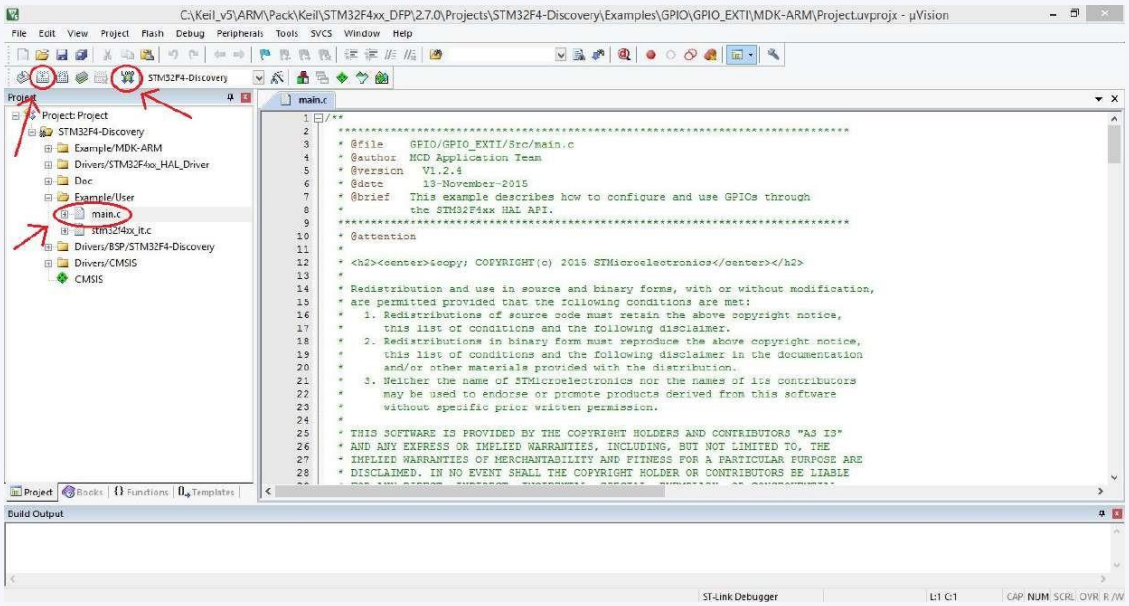


Figure 47 : Icônes Build et Download de Keil µVision 5

Tableau 16 : Récapitulatif des réglages ST-Link

Onglet / fenêtre	Paramètre	Valeur
Debug	Use	ST-Link Debugger
Debug > Settings	Port	SW (Serial Wire)
Flash Download	Algorithme	STM32F4xx Flash, 0x08000000, 1 Mo
Flash Download	Options	Erase Sectors, Program, Verify, Reset and Run
Utilities	Flash Programming	Use Target Driver : ST-Link Debugger

11. Vidéos de démonstration

Les vidéos suivantes illustrent le fonctionnement des montages réalisés. Chaque emplacement est réservé pour insérer la vidéo ou son lien.

1.mp4

Vidéo 2 : Clignotement de la LED externe sur PA5

2.mp4

Vidéo 3 : Feu tricolore sur PA4, PA5 et PA7 (exercice 3)

12. Difficultés rencontrées et solutions

Tableau 17 : Problèmes rencontrés et solutions

Problème	Cause	Solution
« Error #545 : Required input file from generator CubeMX is missing » dans Build Output	Le fichier de configuration de STM32CubeMX n'existe pas avant la première génération.	Configurer les broches dans STM32CubeMX puis cliquer sur GENERATE CODE ; le message disparaît.
Erreur de compilation « symbole défini plusieurs fois » (deux fonctions main)	Le main.c de STM32CubeMX et notre main.c sont compilés ensemble.	Exclure le main.c de STM32CubeMX avec Options for File > décocher Include in Target Build.
« Internal DLL Error » puis « Flash Download failed – Target DLL has been cancelled »	Mauvais débogueur sélectionné (le débogueur par défaut de Keil n'est pas le ST-LINK), ou carte non connectée.	Choisir ST-Link Debugger dans Options for Target > Debug, vérifier le câble USB et le pilote ST-LINK.
La LED ne clignote pas (ou semble toujours allumée)	Broche du code différente de celle de la LED, ou temporisation trop courte.	Utiliser PD12 (LED LD4) comme dans la variante du paragraphe 5.4 et augmenter la boucle de temporisation.
La LED externe (PA5) ne s'allume pas	LED branchée à l'envers, GND non relié, ou fil sur une autre broche que celle du code.	Patte longue côté broche, patte courte vers GND ; vérifier que la broche câblée est bien PA5.
Une LED du feu tricolore reste éteinte	La broche de la LED n'est pas configurée en sortie dans MODER, ou le fil est sur une autre broche.	Contrôler que PA4, PA5 et PA7 sont dans les masques de MODER et que chaque LED est sur la bonne broche.
Téléchargement impossible (ST-Link non détecté)	Câble USB mal branché, pilote absent ou ST-Link Debugger non sélectionné.	Revoir la section 10 : vérifier le Gestionnaire de périphériques puis Options for Target > Debug.

13. Conclusion

Ce chapitre a détaillé la réalisation complète du TP : création du projet sous Keil µVision 5 pour le STM32F407VGTx, configuration de la broche PD12 et de l'horloge dans STM32CubeMX, génération du code, écriture du programme de clignotement en C, puis téléchargement dans la carte par le ST-LINK. Les difficultés rencontrées ont permis de mieux comprendre le rôle de chaque fichier du projet.

Deux exercices complémentaires ont ensuite été réalisés : le clignotement d'une LED externe reliée à PA5, puis un feu tricolore commandé par PA4, PA5 et PA7. Ils ont permis de généraliser la démarche (CubeMX, fichier main.c, ST-Link) à plusieurs broches et d'utiliser les opérations |= et &= ~ pour commander chaque LED indépendamment.

Conclusion générale

Ce TP du module Programmation Embarquées nous a permis de découvrir la carte STM32F407 et la chaîne de développement associée. Nous avons d'abord rappelé les notions de système embarqué et de microcontrôleur, et présenté Arduino (application et cartes) comme point de comparaison. Nous avons ensuite étudié l'architecture du STM32F407, les composants de la carte STM32F4DISCOVERY et le rôle des broches GPIO.

Sur le plan pratique, nous avons installé et utilisé les outils STM32CubeMX, Keil µVision 5 et ST-LINK, créé un projet pour le STM32F407VGTx, configuré une broche en sortie, écrit un programme en C qui manipule directement les registres RCC et GPIO, puis téléchargé le programme dans la carte pour faire clignoter une LED.

Après le clignotement de la LED de la carte, nous avons réalisé deux montages sur breadboard : une LED externe sur PA5 et un feu tricolore sur PA4, PA5 et PA7, avec la configuration détaillée du ST-Link Debugger sous Keil µVision 5.

Ce travail constitue une base solide pour les prochains TP, qui pourront aborder les entrées (bouton poussoir), les interruptions, les timers, la communication série avec Tera Term et la programmation avec la bibliothèque HAL.

Bibliographie

1. Documents et sites de référence

- **Arduino** : téléchargement de l'Arduino IDE : https://www.arduino.cc/en/software ; documentation des cartes : https://docs.arduino.cc/hardware/
- **STMicroelectronics, UM1472** : Discovery kit for STM32F407/417 lines (manuel de la carte) : https://www.st.com/resource/en/user_manual/dm00039084.pdf
- **STMicroelectronics, RM0090** : Reference manual STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 (registres et périphériques) : https://www.st.com/resource/en/reference_manual/dm00031020.pdf
- **STMicroelectronics** : fiche technique du STM32F407VG : https://www.st.com/resource/en/datasheet/stm32f407vg.pdf
- **Keil MDK** : https://www.keil.arm.com/ ; activation de l'édition Community : https://keil.arm.com/mdk-community
- **STM32 Step by Step, Step 1 : Tools installation** : https://wiki.st.com/stm32mcu/wiki/STM32StepByStep:Step1_Tools_installation
- **ST Community** : STM32CubeIDE 2.0, fin du module CubeMX intégré : https://community.st.com/stm32cubeide-mcus-28/stm32cubeide-missing-stm32-project-and-cubemx-after-fresh-installation-162452

2. Bibliothèques et logiciels : où les télécharger et comment les installer

Tableau 18 : Bibliothèques et logiciels utilisés

Élément	Rôle	Où le télécharger	Comment l'installer
Arduino IDE	Éditeur et téléversement des sketchs Arduino	https://www.arduino.cc/en/software	Lancer l'installateur Windows et accepter les pilotes USB.
STM32CubeMX	Configuration des broches et génération du code	https://www.st.com/en/development-tools/stm32cubemx.html	Compte ST > téléchargement > exécuter le Setup en administrateur.
STM32CubeIDE	IDE de ST (Eclipse, GCC, débogueur)	https://www.st.com/en/development-tools/stm32cubeide.html	Compte ST > téléchargement > extraire > lancer l'installateur.
Keil µVision (MDK)	IDE et compilateur ARM	https://www.keil.arm.com/	Télécharger µVision > installer > activer la licence Community.
Pack STM32F4xx_DFP	Fichiers de démarrage et description du STM32F4 pour Keil	Pack Installer de Keil (ou https://www.keil.arm.com/packs/)	Pack Installer > STM32F4 Series > STM32F407 > Install.
ARM::CMSIS (CMSIS-CORE)	Accès normalisé au cœur Cortex-M	Pack Installer de Keil	Pack Installer > ARM::CMSIS > Install ; coché dans Manage Run-Time Environment.
STM32Cube MCU Package for STM32F4 (STM32CubeF4)	Pilotes HAL/LL, fichier stm32f407xx.h	Depuis STM32CubeMX, ou https://github.com/STMicroelectronics/STM32CubeF4	STM32CubeMX > Help > Manage embedded software packages > STM32F4 > Install.

Élément	Rôle	Où le télécharger	Comment l'installer
Pilote ST-LINK (STSW-LINK009)	Reconnaissance du ST-LINK par Windows	https://www.st.com/en/development-tools/stsw-link009.html	Extraire > lancer en administrateur avant de brancher la carte.
Tera Term	Terminal série	https://github.com/TeraTermProject/teraterm/releases	Lancer le fichier setup.exe et suivre l'assistant.